

Yesterday 10:53 PM

Lab_4_VibeCoding...
PDF

Hey, I need to complete a vibe coding lab assignment. I've attached the lab handout and this is my assigned repo: https://github.com/SETAPESU26/20_mastermind also i'm doing the work in VS Code. Can you first go through the README and the existing code and tell me what the lab is asking us to do? I don't want to change anything yet. Just help me understand the current project, what's already there, and what needs to be fixed and added. I am expected to have a before video with all the bugs and an after video with all the bugs fixed with tasks properly shown.

I've gone through the **Lab 4 handout**, the assigned GitHub repo, its README, and all four existing source files. Nothing has been changed.

The important thing is: **your repo is intentionally a small, broken/incomplete Mastermind game**, and the README gives you four distinct tasks to complete.  GitHub

1. What the lab is actually asking you to do

The lab handout says the overall workflow is:

1. Go to your assigned repo.
2. Read the README and understand the deliverables.
3. Clone/fork it.
4. Run the Python game.
5. **Record a 10-second video before making changes**, showing the broken behavior.
6. Use prompts/LLM assistance to fix the code.
7. Add the features in the README **one by one**.
8. Record another 10-second video after everything works.
9. Push your code.
10. Make **separate commits for each task**.
11. Do **not** make a PR to the main **SETAPESU26** repo.  Lab_4_VibeCoding_Student_handout

The handout also specifically says this is an **individual assignment**, with a 60-minute duration.



Lab_4_VibeCoding_Student_handout

Your repo's README then defines exactly what the four tasks are.

2. What you currently have

Your repository is extremely small:

</> Plain text

```
20_mastermind/  
├─ README.md  
├─ requirements.txt  
├─ main.py  
├─ game.py  
└─ logic.py
```

The README describes it as:

A terminal code-breaking game with repeated symbols and structured feedback.

It has:

- `main.py` → entry point
- `game.py` → game loop and history
- `logic.py` → exact/partial match calculation
- `requirements.txt` → dependencies [GitHub](#)

There are **no external packages** required. `requirements.txt` explicitly says it is standard-library-only and recommends Python 3.9+. [GitHub](#)

So this is nice for your lab: you shouldn't need to install a bunch of things.

3. How the current program works

`main.py`

It's basically just the launcher:

</> Python

```
from game import Mastermind  
  
if __name__ == "__main__":  
    Mastermind().run()
```

So when you execute:

</> Bash

`python main.py`

it creates a `Mastermind` object and starts the game. [GitHub](#)

There isn't really a bug to fix here.

`game.py`

This is where most of the game behavior lives.

Currently it:

- randomly generates a **4-character code**
- uses digits **1–6**
- gives the player **10 turns**
- accepts a 4-digit guess
- calculates feedback
- stores guesses in `history`
- decreases the turn count
- declares a win if there are 4 exact matches
- reveals the code when the turns run out. [GitHub](#)

The current setup is hard-coded here:

`</>` Plain text

```
Code length = 4
Symbols = 1-6
Turns = 10
```

There are no difficulty modes yet.

4. The biggest existing bug: duplicate handling

This is **Task 1**, and this is almost certainly the bug you want to demonstrate in your before video.

Current `logic.py` does this:

`</>` Python

```
exact = sum(a == b for a, b in zip(code, guess))
partial = sum(ch in code for ch in guess) - exact
```

The README explicitly warns you about this:

"the code, so duplicate symbols can consume the same code occurrence."  GitHub

The problem is that:

</> Python

```
ch in code
```

only asks:

"Does this character exist anywhere in the code?"

It does **not** keep track of how many times that character is available.

Example

Suppose the secret code is:

</> Plain text

```
1234
```

and the player guesses:

</> Plain text

```
1111
```

There is only **one** **1** in the code.

The correct result should therefore be:

</> Plain text

```
Exact: 1
```

```
Partial: 0
```

But the current algorithm effectively sees four **1**s that are "in the code":

</> Plain text

```
1 → exists
```

```
1 → exists
```

```
1 → exists  
1 → exists
```

and then subtracts the one exact match.

So it reports:

</> Plain text

```
Exact: 1  
Partial: 3
```

That's wrong.

An even more obvious repeated-symbol example is:

</> Plain text

```
Code: 1122  
Guess: 1111
```

Correct:

</> Plain text

```
Exact: 2  
Partial: 0
```

because the two **1**s in the code have already been consumed by the two exact matches.

The current implementation can report:

</> Plain text

```
Exact: 2  
Partial: 2
```

which is exactly the kind of defect the README wants you to reproduce and fix.

The README's requirement is very specific:

Each code position can be counted at most once, and exact matches must be resolved before partial matches. 

5. Task 1 — Correct duplicate-aware feedback

What you need to achieve

Fix the feedback algorithm so that:

1. Exact matches are counted first.
2. Those positions are no longer available for partial matches.
3. Each remaining code symbol can contribute to a partial match **only once**.
4. Repeated symbols work correctly.

The README says Task 1 is done when:

repeated-symbol cases produce the correct exact and partial counts. [GitHub](#)

What you should demonstrate before changing anything

For your **before video**, this is probably the best bug to intentionally show.

You can use a repeated-symbol case that produces visibly incorrect feedback.

You don't need to change the code to demonstrate it. The current code already has the defect.

6. Task 2 — Complete game lifecycle

This is bigger than just fixing the feedback calculation.

The README says you need to:

- add clear **win handling**
- add clear **loss handling**
- preserve guess history
- enforce the guess limit
- prevent state changes after the game ends. [GitHub](#)

There are several things to notice about the current implementation.

History technically exists

`game.py` already has:

```
</> Python
```

```
self.history = []
```

and every accepted guess gets appended:

</> Python

```
self.history.append((raw, exact, partial))
```

So the groundwork is there.

But the player never actually sees the history.

The program simply prints the latest:

</> Plain text

Exact: ...

Partial: ...

It doesn't provide a readable list/table of previous guesses.

So "history" exists internally but isn't really a user-facing feature yet.

Win handling

Currently:

</> Python

```
if exact == 4:  
    print("Cracked the code!")  
    return
```

So a win does stop the current `run()` loop.

That's partially implemented already.

But the README wants the **whole lifecycle** handled cleanly, including preventing state changes after the game has ended.

Loss handling

When the turns run out, it does:

</> Python

```
print("The code was", "".join(self.code))
```

So there is a basic loss outcome.

But it isn't really a complete/structured game lifecycle yet.

You'll need to make the end states explicit and make sure the game cannot continue or mutate after either outcome.

7. Task 3 — Difficulty modes

This feature doesn't exist at all right now.

The README says to add difficulty choices that change:

- **code length**
- **symbol range**
- **number of allowed guesses**

And importantly:

The feedback algorithm must work for all supported configurations.  [GitHub](#)

So you will need some sort of difficulty selection when starting the game.

Conceptually, you're moving from:

</> Plain text

```
Every game:  
4 digits  
1-6  
10 guesses
```

to something like:

</> Plain text

```
Choose difficulty  
  ↓  
difficulty determines  
  ↓  
code length  
symbol range  
guess limit
```

The README does **not** prescribe specific difficulty names or exact values.

That's important: **we shouldn't invent requirements that aren't actually stated.**

When we get to the implementation stage, we can choose sensible values while keeping the implementation simple.

8. Task 4 — History and input robustness

This task has two major pieces.

A. Readable history

Instead of only showing the latest result, the player should be able to see something along the lines of:

</> Plain text

Guess	Exact	Partial
1234	2	1
5612	1	2
...		

The exact display format isn't specified by the README; the requirement is that the history be **readable**.

B. Invalid guesses must not consume turns

The current program actually has a decent start here:

</> Python

```
if len(raw) != 4 or any(ch not in "123456" for ch in raw):  
    print("Enter exactly four digits from 1 to 6.")  
    continue
```

Because of the `continue`, an invalid guess does **not** get added to history and does **not** decrement `self.turns`.

So some of the input robustness already exists.

However, it is hard-coded to:

</> Plain text

4 characters

1-6

which won't work once Task 3 introduces different difficulty configurations.

Therefore Task 4 and Task 3 are related.

For example, if a difficulty has a 5-symbol code, the input validation must understand that.

9. Required testing

This is particularly important because it gives you your **after-video/testing checklist**.

The README explicitly says you need to test:

- exact matches
- repeated symbols
- no matches
- mixed exact/partial cases
- every difficulty
- invalid guesses
- final-turn wins
- losses
- quitting.  GitHub

So don't just get the game to "look like it works."

You want to be able to demonstrate these cases.

10. Your current project vs. final project

Here's the simplest way to think about your assignment:

Area	Current state	What final version needs
Basic game	✓ Exists	Keep working
Random code	✓ Exists	Work across difficulties
Exact matching	✓ Exists	Keep correct
Partial matching	✗ Buggy	Duplicate-aware

Area	Current state	What final version needs
Repeated symbols	✗ Incorrect	Correct
Win	⚠ Basic	Complete lifecycle
Loss	⚠ Basic	Clear lifecycle
Turn limit	⚠ Basic	Properly enforced
Post-game state	⚠ Not explicitly protected	Prevent changes
History storage	✓ Exists	Preserve
History display	✗ Missing	Readable
Difficulty	✗ Missing	Add
Input validation	⚠ Partially exists	Work with all difficulties
Invalid guess consumes turn	✓ Currently prevented	Keep this behavior
Quit	✓ Exists	Keep working
Persistence	✗	Do not add
External dependencies	None	Avoid unnecessary ones

The README also explicitly says **not to add CSV, JSON, SQLite, or other persistence** and to keep all state in memory. [GitHub](#)

11. What I would do for your BEFORE video

Since you're expected to have a 10-second before video showing the bug/broken behavior, I would not try to show everything.

The strongest demonstration is the duplicate bug.

Before video target

Run:

```
</> Bash
```

`python main.py`

Then deliberately enter a repeated-symbol scenario that exposes the incorrect feedback.

For example, you want the secret code/guess situation to demonstrate:

</> Plain text

A symbol appears fewer times in the secret
than it appears in the guess

and show that the program counts the symbol too many times.

You can also briefly show that:

- there is no difficulty selection
- previous guesses aren't displayed as history

But **the duplicate feedback bug should be the centerpiece**, because Task 1 specifically says the original defect must be reproduced and fixed.

The README even instructs you, before changing the code, to construct codes and guesses containing repeated symbols and reason about how many times one code occurrence should contribute.

 GitHub

12. Your commits should correspond to the tasks

The lab handout explicitly says:

"Have Separate Commits for each task."



Lab_4_VibeCoding_Student_handout

So I would structure your work as:

</> Plain text

Task 1 → duplicate-aware feedback
Task 2 → game lifecycle
Task 3 → difficulty modes
Task 4 → history + robust input

Ideally your Git history makes that progression obvious.

That is also good for the "vibe coding" aspect because you can show that you didn't just ask an AI to rewrite the whole project—you worked through the requested tasks individually.

13. One important thing about the videos

The lab handout says the submission should include **before and after videos, updated code, and exported chat history**.  [Lab_4_VibeCoding_Student_handout](#)

Your repo README is even more specific: its submission checklist says the three things are:

1. **10-second before video** showing the broken behavior
2. **10-second after video** showing the bug fixed + new features
3. **Chat/LLM page link with complete chat history** 

So for this lab, **your ChatGPT conversation is actually part of the evidence**. That's useful because we should make our later prompts/task progression clean and understandable.

The overall picture

Think of the assignment as:

 Plain text

CURRENT REPO

```
|  
├─ Bug 1: duplicate feedback  
|  
├─ Missing: proper game lifecycle  
|  
├─ Missing: difficulty modes  
|  
└─ Missing/weak: history + robust input  
|  
▼
```

TASK 1

Fix feedback algorithm

```
|  
▼
```

TASK 2

Make game lifecycle complete

```
|  
▼
```

TASK 3

And importantly, **you do not need to redesign this project**. The README specifically says not to replace the whole project with an unrelated implementation. 

What I recommend we do next

Before touching any code, the next useful step is to **run the existing project exactly as-is and reproduce the bugs**, so we know what your actual before-state looks like.

If you're ready, I can walk you through **exactly what to type in VS Code's terminal**, what behavior to look for, and how to capture the 10-second before video—**without changing a single line of code**.

okay I understood all of it... its simple

Exactly 😊 — it's actually a pretty manageable lab once you break it down.

The main thing now is **not to overcomplicate it**. We already know the four tasks and what the expected final behavior should be.

When you're ready, we can go step-by-step:

1. Run the untouched repo
2. Reproduce the duplicate-symbol bug
3. Record your before video
4. Then tackle Task 1 → Task 2 → Task 3 → Task 4, one prompt/commit at a time
5. Test everything
6. Record the after video
7. Push your repo and make sure your submission evidence is clean

And I'll help you keep the **ChatGPT prompt history aligned with the four separate tasks**, which is useful since the lab specifically asks for the chat history as part of the submission.

 Lab_4_VibeCoding_Student_handout

Whenever you're ready, say **"let's run the repo"** and we'll start with the untouched version.

okay lets go ahead step by step

Step 1 — Open and run the untouched repo

In **VS Code**, open your cloned `20_mastermind` folder.

1. Open the terminal

Use:

Terminal → New Terminal

Then check that you're inside the project folder. You should see something roughly like:

 Plain text

```
...\20_mastermind>
```

2. Run the existing game

Use:

```
</> Bash
```

```
python main.py
```

If that doesn't work, try:

```
</> Bash
```

```
python3 main.py
```

Do not install anything or modify any files yet.

3. Play it normally once

Just observe what happens:

- Does the game start?
- What does it ask you for?
- How many attempts do you get?
- What happens after a valid guess?
- What happens after an invalid guess?
- What happens if you enter `q` ?
- What happens when you win?
- What happens when you run out of attempts?

We're establishing the baseline.

4. Then reproduce the duplicate-symbol bug

This is the important part.

Because the secret code is randomly generated, we need to **see the secret code** so that we can deliberately construct a duplicate-symbol test.

The easiest way without changing the code is to let the game eventually reveal the code after you exhaust the attempts, **or inspect the existing behavior/code to construct an appropriate test case.**

For your before video, we want something that demonstrates:

The same occurrence of a symbol is being counted more than once as a partial match.

For example, if the secret were:

</> Plain text

1122

and the guess were:

</> Plain text

1111

the correct result is:

</> Plain text

Exact: 2

Partial: 0

because the secret contains only two **1** s.

The current implementation can incorrectly count additional **1** s as partial matches.

Don't record yet

First run it and tell me **exactly what you see in the terminal**.

If you get an error, paste the error here instead.

Once you've successfully run it, we'll do the duplicate-symbol test together and I'll tell you **exactly what to put in your 10-second before video**.

I ran it once, I understood how it works..

Perfect. 👍

Now we know the **untouched baseline works enough to launch and play**, so we can move to the actual bug reproduction.

Step 2 — Reproduce the duplicate-symbol bug

Before changing anything, we want to intentionally trigger the bug from **Task 1**.

The cleanest way is to use the existing `logic.py` function directly, because the bug is in the feedback calculation itself. We can test it without modifying the project.

From the VS Code terminal, run:

</> Bash

```
python
```

Then enter:

</> Python

```
from logic import score  
score("1122", "1111")
```

You should get the current implementation's result.

Send me exactly what it prints.

Don't change any files yet. Once we confirm the incorrect result, I'll show you the next tiny test and then we'll record the **before video**.

we need to first look for bugs, can we use a debugger.. set a breakpoint if that works?

Absolutely. In fact, **using VS Code's debugger is a better approach for this lab** because we're supposed to understand the existing broken code before fixing it.

We can inspect the bugs without changing anything.

Let's do this first: inspect `logic.py`

Open:

</> Plain text

```
20_mastermind/  
└─ logic.py
```

Find the function that calculates the `exact` and `partial` matches.

Set a breakpoint on the line where `exact` is calculated:

</> Python

```
exact = ...
```

In VS Code:

1. Click in the **left margin** beside that line.
2. A red dot should appear.
3. Press **F5** or go to **Run → Start Debugging**.
4. Choose **Python File** if VS Code asks.

Then we'll be able to inspect:

- `code`
- `guess`
- `exact`
- `partial`

and, more importantly, **step through the algorithm line by line**.

One thing before we do that

Since the game generates the code randomly, I don't want you to randomly play and hope we hit the bug.

We'll use the debugger to deliberately give the scoring function a controlled example such as:

</> Plain text

```
code = "1122"  
guess = "1111"
```

Then we'll watch exactly what the current algorithm does.

That gives us a very clean understanding of **why** the bug happens, rather than just observing that the output is wrong.

So yes — **let's use the debugger**.

Open `logic.py`, set the breakpoint on the `exact = ...` line, and tell me when you see the red breakpoint dot.

what if i set a debugger in game.py?

Yes — `game.py` is actually a good place to start with the debugger, because it lets us see the whole game flow before drilling down into the scoring bug.

I'd use it in two stages:

1. First breakpoint: `game.py`

Set a breakpoint around the line where the user's guess is passed to the scoring function.

It should be around the part that does something like:

</> Python

```
exact, partial = score(self.code, raw)
```

Then start the game with **F5**.

When execution stops there, look at the **Variables** panel.

We want to inspect:

</> Plain text

```
self.code  
raw  
self.turns  
self.history
```

This tells us exactly what the game generated and what the user entered.

2. Step into the scoring function

This is the useful part.

When the debugger stops on the `score(...)` call, use **Step Into (F11)**.

VS Code should take you from:

</> Plain text

```
game.py  
  ↓  
logic.py  
  ↓  
score(...)
```

Then we can watch the faulty calculation happen.

So the debugging flow becomes:

</> Plain text

```
game.py  
  |  
  | user enters guess  
  ▼  
score(self.code, raw)  
  |
```

```
| F11 / Step Into
▼
logic.py
|
|— calculate exact
|— calculate partial ← BUG IS HERE
|
▼
return feedback
|
▼
game.py displays result
```

This is better than immediately starting in `logic.py`, because you'll understand how the bug travels through the actual game.

And importantly, we're still making **zero changes**.

Set the breakpoint at the `score(...)` call in `game.py`, run with **F5**, and tell me when the debugger pauses there. Then I'll guide you through the variables and F11 one step at a time.

okay understood

Great. 👍

We'll continue exactly this way: **understand** → **reproduce** → **identify** → **fix** → **test** → **commit**, one task at a time.

For now, don't change anything. Once you're at the breakpoint in `game.py`, we'll inspect the values and step into `logic.py` to see the bug happen.

the secret key is 5 3 3 3

Perfect. That secret code is:

</> Plain text

5333

This is actually a **great test case for the duplicate-symbol bug** because `3` occurs **three times**.

Let's use the debugger to expose exactly what the current code does.

In the running game

Enter this guess:

</> Plain text

3333

The correct feedback should be:

</> Plain text

Exact: 3

Partial: 0

Why?

</> Plain text

Secret: 5 3 3 3

Guess: 3 3 3 3

↑ ↑ ↑

exact matches

There are three 3 s available in the secret, and all three are already consumed by the exact matches. The fourth guessed 3 shouldn't count as a partial match.

Now use the debugger

When execution reaches the `score(...)` line in `game.py`:

1. Look at `self.code` → should be `5333`
2. Look at `raw` → should be `3333`
3. Press **F11 (Step Into)**
4. You should enter `logic.py`
5. Step through the `exact` calculation.
6. Then watch what happens when the `partial` calculation runs.

Don't fix anything yet.

Tell me what result the program gives you for `5333` vs `3333`, and we'll use that to identify the bug precisely.

when I used 3333 i got exact:3 and partial: 1 whereas when i used 5333 the output says exact:4 and partial: 0 cracked the code

Exactly — you've reproduced the bug. 🎯

Your secret is:

</> Plain text

5333

and you entered:

</> Plain text

3333

The program gave:

</> Plain text

Exact: 3

Partial: 1

But the correct result is:

</> Plain text

Exact: 3

Partial: 0

Why the current code is wrong

The three 3 s that match exactly have already used up all three 3 s available in the secret:

</> Plain text

Secret: 5 3 3 3

Guess: 3 3 3 3

✓ ✓ ✓

The fourth 3 has **nothing left to match**.

The current algorithm doesn't track that consumption. It essentially asks whether each guessed character exists somewhere in the code, which causes that fourth 3 to be incorrectly counted as a partial match.

And your second test:

</> Plain text

```
Secret: 5333
Guess: 5333

Exact: 4
Partial: 0
Cracked the code!
```

is correct, so we've isolated the problem nicely.

🎥 This is your before-video evidence

This is actually an excellent case to show:

1. Start the game.
2. Show the secret is `5333` if it is visible/revealed during your debugging demonstration.
3. Enter `3333`.
4. Show:

</> Plain text

```
Exact: 3
Partial: 1
```

5. Explain that the expected result is `Exact: 3, Partial: 0`.

Don't fix it yet. We have now successfully demonstrated Task 1's broken behavior.

Next, I'd like you to **step through** `logic.py` with F11 for this exact `5333 / 3333` case. That will let us see *the precise line and variable causing the* `partial: 1`, which is useful both for understanding the code and for your vibe-coding prompt later.

we need to find the bugs firsts so that we can add all of it in the before video.. as per the readme file we need to check for duplicates, Add clear win/loss handling, preserve guess history, enforce the guess limit, and prevent state changes after the game ends, add difficulty modes and check for history and input robustness.. lets check for these bugs first

Worked for 13s

Yes. **You're right to stop fixing anything and first map the complete before-state.** The README explicitly tells us to inspect the existing code first and test the stated behaviors/edge cases.  GitHub

I checked the current `game.py` and `logic.py` again. Some requirements are **already partially implemented**, so we should not falsely call everything a bug. We should identify:

-  genuinely broken

- 🚧 partially implemented / missing required behavior
- ✅ already working

That will give us a clean before-video plan.

Before-state investigation

The README gives us these four areas:

1. Duplicate-aware feedback
2. Complete game lifecycle
3. Difficulty modes
4. History + input robustness

And it explicitly requires testing exact matches, repeated symbols, no matches, mixed matches, every difficulty, invalid guesses, final-turn wins, losses, and quitting.  [GitHub](#)

Let's go through them **without changing code**.

1. Duplicate handling — ❌ CONFIRMED BUG

We've already found this one.

Your secret:

</> Plain text

5333

Your guess:

</> Plain text

3333

Current result:

</> Plain text

Exact: 3

Partial: 1

Correct result:

</> Plain text

Exact: 3

Partial: 0

The current implementation is:

</> Python

```
exact = sum(a == b for a, b in zip(code, guess))
partial = sum(ch in code for ch in guess) - exact
```

The problem is that `ch in code` doesn't consume an occurrence. [GitHub](#)

Status

✖ Definitely broken.

This is our strongest before-video bug.

2. Win handling — ⚠ PARTIALLY IMPLEMENTED

Look at `game.py`:

</> Python

```
if exact == 4:
    print("Cracked the code!")
    return
```

So there is already a win condition. [GitHub](#)

We shouldn't say "win handling doesn't exist."

But the assignment specifically asks for **complete game lifecycle** and prevention of state changes after the game ends. [GitHub](#)

Let's test it.

You already did:

</> Plain text

5333

and:

```
</> Plain text
```

```
5333
```

gave:

```
</> Plain text
```

```
Exact: 4
```

```
Partial: 0
```

```
Cracked the code!
```

Now observe what happens after that.

The program returns from `run()`, so it **does stop accepting guesses**.

Status

⚠️ Basic win handling already exists.

We need to determine during Task 2 whether it satisfies the full lifecycle requirement or needs to be made more explicit/structured.

3. Loss handling — ⚠️ BASIC IMPLEMENTATION EXISTS

The current code ends with:

```
</> Python
```

```
print("The code was", "".join(self.code))
```

after:

```
</> Python
```

```
while self.turns:
```

finishes.  GitHub

So there is a **loss outcome**.

We need to test whether the full behavior is correct:

Test

Don't try to win.

Enter **10 valid but incorrect guesses**.

Watch:

</> Plain text

```
10 turns left
9 turns left
8 turns left
...
1 turn left
```

After the 10th accepted guess, it should reveal:

</> Plain text

```
The code was XXXX
```

and stop.

Things we want to observe

- Does it stop after exactly 10 accepted guesses?
- Does the 10th guess get processed?
- Does it reveal the code?
- Does it ask for an 11th guess?

Status

⚠ Basic loss handling appears to exist.

We need to verify it with the debugger/game.

4. Guess limit — CURRENTLY MOSTLY WORKING

This is important.

Current code:

</> Python

```
while self.turns:
```

starts with:

```
</> Python
```

```
self.turns = 10
```

and after every valid guess:

```
</> Python
```

```
self.turns -= 1
```

So the existing implementation already enforces **10 accepted guesses**. [GitHub](#)

Also notice this:

```
</> Python
```

```
if len(row) != 4 or any(ch not in "123456" for ch in row):  
    print(...)  
    continue
```

The `continue` happens **before**:

```
</> Python
```

```
self.turns -= 1
```

So an invalid guess currently does **not** consume a turn.

That's actually correct behavior according to Task 4.

Status

✅ Already implemented for the current fixed 4-digit game.

But Task 3 will change the code length/range/guess limit, so this hard-coded implementation will need to become configurable.

5. State changes after game ends — ⚠️ NEED TO VERIFY

The current code uses:

```
</> Python
```

```
return
```

after:

```
</> Python
```

```
if raw.lower() == "q":
```

and after winning.  GitHub

When turns reach zero, the `while` loop ends.

So at first glance, the game **does stop changing state after the game ends**.

But let's test this rather than assume.

Debugger test

Set a breakpoint around:

```
</> Python
```

```
self.turns -= 1
```

Then win with:

```
</> Plain text
```

```
5333
```

When it reaches:

```
</> Plain text
```

```
Cracked the code!
```

execution should leave `run()`.

There should be no subsequent guess processing.

Status

⚠️ Appears mostly implemented already.

This may become part of Task 2's cleanup rather than being a completely broken feature.

6. Guess history — ❌ USER-FACING HISTORY IS MISSING

This one is clear.

The code has:

Python

```
self.history = []
```

and then:

Python

```
self.history.append((raw, exact, partial))
```

So history is actually being **stored**.  [GitHub](#)

But where is it displayed?

Nowhere.

The only feedback shown is:

Python

```
print("Exact:", exact, " Partial:", partial)
```

So if you make:

Plain text

```
1234  
5612  
4444
```

you don't get a table/list such as:

Plain text

Guess	Exact	Partial
1234	1	2
5612	0	1
4444	2	0

Debugger test

After making 2–3 valid guesses, pause at:

</> Python

```
self.history.append(...)
```

and inspect:

</> Plain text

```
self.history
```

You'll see the data is being preserved internally.

Then let the game continue.

The player **never sees that history**.

Status

✗ Required readable history is missing.

This is definitely something we can show as a before-state deficiency.

7. Difficulty modes — ✗ COMPLETELY MISSING

This one is straightforward.

Current constructor:

</> Python

```
self.code = [str(random.randint(1, 6)) for _ in range(4)]  
self.turns = 10
```

Everything is hard-coded:

</> Plain text

```
code length = 4
symbols = 1-6
turns = 10
``` :chatgpt-content-reference{index="9"}
```

There is:

- no difficulty menu
- no difficulty selection
- no easy/medium/hard configuration
- no configurable code length
- no configurable symbol range
- no configurable number of guesses

The README explicitly requires difficulty choices that vary all three. :chatgpt-content-reference{index="

### Status

\*\*  Completely missing \*\*

So these should be rejected:

**</> Plain text**

```
123
12345
abc4
7891
12a4
```

And because of:

**</> Python**

```
continue
```

they shouldn't consume a turn.  GitHub

That's actually aligned with Task 4:

reject malformed guesses without consuming turns.  GitHub

**Let's verify.**

Start a fresh game.

Suppose it says:

</> Plain text

10 turns left >

Enter:

</> Plain text

123

You should see the error.

Then check:

</> Plain text

10 turns left >

If it still says 10, that's correct.

Then try:

</> Plain text

9999

Again, it should remain at 10.

## Status

✓ Current basic validation works.

But it is hard-coded for:

</> Plain text

4 digits

1-6

Once difficulty modes exist, this validation must become dynamic.

## 9. "Feedback corresponds to one accepted guess only"

This is another requirement we should test.

Current flow is:

</> Plain text

```
input
↓
validate
↓
feedback()
↓
history.append()
↓
turns -= 1
↓
print feedback
```

Invalid input exits through `continue`, so it shouldn't reach `feedback()`.  [GitHub](#)

Therefore this currently looks correct.

### Status

✅ Appears correct for the current fixed configuration.

We'll retest after Task 3 because changing difficulty can easily break this.

## 10. Quit behavior — ✅ EXISTS

The code explicitly has:

</> Python

```
if raw.lower() == "q":
 return
```

So:

</> Plain text

q

or:

&lt;/&gt; Plain text

Q

ends the game.  GitHub

The README specifically requires quitting to be tested.  GitHub

## Status

✓ Already implemented.

## So here's our REAL before-state

This is the list I'd keep in front of us while testing:

Requirement	Current state
Duplicate-aware feedback	✗ BUG
Exact matches	✓ Works
No-match feedback	● Need to test
Mixed exact/partial	● Need to test
Win handling	⚠ Basic implementation
Loss handling	⚠ Basic implementation
Guess limit	✓ 10-turn limit works
No state changes after end	⚠ Appears protected
Guess history stored	✓ Internally
Guess history displayed	✗ Missing
Difficulty selection	✗ Missing
Variable code length	✗ Missing
Variable symbol range	✗ Missing

Requirement	Current state
Variable guess limit	✗ Missing
Invalid input rejected	✓ Current fixed config
Invalid input doesn't consume turn	✓
One feedback per accepted guess	✓ Appears correct
Quit	✓
Difficulty-specific testing	✗ Impossible currently

The README itself makes these requirements explicit: Task 1 is duplicate-safe feedback; Task 2 is lifecycle/history/limits; Task 3 is configurable difficulty; Task 4 is readable history and robust input.



## Now let's systematically test the remaining BEFORE behavior

I suggest we don't record the before video yet.

Let's first gather evidence for every required case.

### Test A — No matches

Use a guess containing symbols completely absent from the secret.

For your current secret:

</> Plain text

5333

a good example is:

</> Plain text

1111

Expected:

</> Plain text

Exact: 0  
Partial: 0

Tell me what the program gives you.

## Test B — Mixed exact + partial

For:

</> Plain text

Secret: 5333

try:

</> Plain text

3355

We'll calculate the expected result and compare it with the program.

## Test C — History

Make 2–3 valid guesses and observe whether previous guesses are displayed.

Also inspect `self.history` in the debugger if you want to verify that it is being stored.

## Test D — Invalid input

Try:

</> Plain text

123

and then:

</> Plain text

9999

Check whether the turn count changes.

## Test E — Loss

Use 10 valid incorrect guesses and verify exactly what happens on the 10th.

## Test F — Quit

Enter:

</> Plain text

q

and verify that the game stops.

## Test G — Difficulty

We already know there is **no difficulty selection**, but we'll document that as a missing feature rather than try to "test" something that doesn't exist.

Let's do Test A first: with the current secret **5333**, enter **1111** and tell me the output. Then we'll work through  $B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$ , and by the end we'll have a complete, evidence-based before-state checklist for your video.

okay

Great. Let's do Test A — no matches.

Your current secret is:

</> Plain text

5333

Enter:

</> Plain text

1111

Then tell me exactly what the game displays for:

</> Plain text

Exact: ?

Partial: ?

We'll compare it with the correct expected result and move to **Test B — mixed exact/partial**.

the new secret key is 2 2 5 2

No problem. The new secret is:

</> Plain text

2252

Let's continue **Test A — no matches**.

Enter:

</> Plain text

1111

For the secret 2252, the correct result should be:

</> Plain text

Exact: 0

Partial: 0

because there are no 1 s anywhere in 2252 .

Tell me what the game actually displays. Then we'll move to the mixed exact/partial test.

it shows exact and partial both 0

Good.  That confirms the **no-match case works correctly**.

For:

</> Plain text

Secret: 2252

Guess: 1111

we got:

</> Plain text

Exact: 0

Partial: 0

So we should not list "no matches" as a **bug** in the before video.

### Test B — Mixed exact + partial

Let's now test a case where we expect **both** exact and partial matches.

With your secret:

</> Plain text

2252

enter:

</> Plain text

2522

Let's calculate it before looking at the output:

</> Plain text

Secret: 2 2 5 2

Guess: 2 5 2 2

↑

exact

- Position 1: 2 → exact
- The remaining secret symbols are 2, 5, 2
- The remaining guess symbols are 5, 2, 2
- 5 can match once
- one of the remaining 2 s can match once

So the **correct** result should be:

</> Plain text

Exact: 1  
Partial: 3

Actually, let's be careful here: after the one exact **2**, the remaining secret is **2,5,2** and remaining guess is **5,2,2**, so all three remaining symbols can indeed be matched — **1 exact + 3 partial**.

Run **2522** and tell me what the game reports.

i got exact 2 and partial 2

Good — and this reveals something important. **My expected result in the previous message was wrong.** We should use the actual scoring rules in the README/code, not assume the result.

For:

</> Plain text

Secret: 2252  
Guess: 2522

Let's calculate it carefully.

</> Plain text

Position:	1	2	3	4
Secret:	2	2	5	2
Guess:	2	5	2	2
	✓			✓

There are **2 exact matches**: positions 1 and 4.

After removing those exact positions:

</> Plain text

Remaining secret: 2 5  
Remaining guess: 5 2

Both remaining symbols match, but in different positions:

</> Plain text

5 ↔ 5

2 ↔ 2

So the correct result is:

</> Plain text

Exact: 2

Partial: 2

Therefore Test B passes 

Your program gave:

</> Plain text

Exact: 2

Partial: 2

which is correct.

So far our evidence is:

Test	Result
Duplicate-heavy: 5333 vs 3333	 Bug: 3 exact, 1 partial
No matches: 2252 vs 1111	 Correct
Mixed exact/partial: 2252 vs 2522	 Correct

This is actually useful for the before video: **the feedback algorithm isn't completely broken; its specific weakness is duplicate counting.**

**Next: Test C — Guess history**

Now make **2–3 valid guesses** in the current game.

Don't worry about winning. For example, enter:

</> Plain text

1111

1234

4444

After each guess, look at the terminal.

**Question:** Does the game display your previous guesses and their Exact/Partial results, or does it only show the result for the latest guess?

Tell me what you see.

only latest guess

Exactly. That confirms another requirement.

Test C — History result

You made multiple valid guesses, but the terminal only showed the **latest** result.

So:

- `self.history` is being stored internally 
- **Readable history for the player is missing** 

This is a real Task 4 deficiency.

For the before video, we can show something simple like:

`</>` Plain text

Guess 1 → feedback  
Guess 2 → feedback  
Guess 3 → feedback

and then point out that only the third/latest result is visible.

Current bug checklist

We're building this up nicely:

Requirement	Before-state result
Duplicate handling	 Bug confirmed
No matches	 Works
Mixed exact/partial	 Works

Requirement	Before-state result
Win handling	⚠ Basic — need full lifecycle test
Loss handling	⚠ Need to test
Guess limit	⚠ Need to verify 10th/final turn
State after game ends	⚠ Need to test
Guess history storage	✅ Exists internally
Readable guess history	❌ Missing
Difficulty modes	❌ Missing
Invalid input	⚠ Need to test
Invalid input consumes turn?	⚠ Need to verify
Quit	⚠ Need to test

We haven't fixed **anything**, which is exactly what we want at this stage.

### Next: Test D — invalid input

Start/continue a game and enter:

</> Plain text

123

Then look at the turn count.

After that, enter:

</> Plain text

9999

Again, check whether the turn count decreases.

We want to establish whether invalid guesses are correctly rejected **without consuming a turn**.

Tell me what happens with **123** first.

it says that it needs 4 digit and the turn remains same ie 5

Perfect. That confirms another **working behavior**, so we won't call it a bug.

For:

</> Plain text

Input: 123

the game:

- rejected it because it isn't 4 digits ✓
- kept the turn count at 5 ✓

So the current input validation correctly prevents malformed input from consuming a turn.

## Continue Test D

Now enter:

</> Plain text

9999

This is 4 digits, but the allowed symbols are only 1-6 .

We want to see whether it:

1. rejects 9999
2. keeps the turn count at 5

Tell me what happens.

it says to enter exactly 4 between 1 to 6 and still has turn 5

Exactly. ✓

So **invalid-input handling is currently working correctly** for the existing fixed configuration.

We have now confirmed:

- 123 → rejected, turn stays at 5 ✓
- 9999 → rejected, turn stays at 5 ✓

This will matter later when we add difficulty modes: the same robustness needs to continue working with whatever code length/symbol range each difficulty uses.

## Current before-state

We can now confidently mark:

Requirement	Status
Duplicate handling	✗ Bug confirmed
No matches	✓
Mixed exact/partial	✓
History displayed	✗ Missing
Invalid input rejected	✓
Invalid input doesn't consume turn	✓
Difficulty modes	✗ Missing

Now let's investigate the **game lifecycle**, which is the remaining major area.

## Test E — Final-turn/loss behavior

Let's deliberately lose a game.

You currently have **5 turns remaining**, so you can either:

- continue this game with five valid guesses that definitely don't crack the code, **or**
- restart with `python main.py` to get a fresh 10-turn game.

I'd recommend restarting so we have a clean test.

Run:

```
</> Bash
```

```
python main.py
```

Then make **10 valid guesses without trying to solve the code**.

We're specifically watching:

1. Does the counter reach 0?
2. Is the 10th guess processed?
3. Does the game reveal the secret?
4. Does it stop accepting guesses after that?

Don't worry about the exact feedback for these guesses.

Tell me what happens when you reach the **10th and final guess**.

i used the same, I reached the final turn and maybe i guessed wrong so i got the code specifically displayed that it is 2252

Perfect. That confirms the **loss path is working at a basic level**. ✓

When you used all 10 turns without cracking the code, the game:

- accepted the final valid guess
- reached the end of the allowed turns
- revealed the secret code 2252
- stopped the game

So we should **not claim that loss handling or the 10-turn limit is completely broken**.

## Our evidence so far

Requirement	Result
Duplicate symbols	✗ Bug — incorrect partial count
No matches	✓ Works
Mixed exact/partial	✓ Works
Readable guess history	✗ Missing
Invalid length input	✓ Works
Invalid symbols	✓ Works
Invalid input consumes turn	✓ Correctly prevented
Loss after final turn	✓ Basic behavior works
Secret revealed on loss	✓ Works
Difficulty modes	✗ Missing

There are still **two important lifecycle checks** before we finish the before-state.

## Test F — Can the game continue after a loss?

Since you just reached the loss state, try entering another guess **if the program is still accepting input**.

If it has already returned to the terminal prompt, that's actually evidence that the game stopped correctly.

Tell me what happens.

Then we'll test the **win state** in the same way: win the game and see whether anything can happen after `"Cracked the code!"`.

After that we'll investigate the difficulty requirement and we'll have a complete list of what belongs in your before video.

since i did not win or guess correctly it returned to the terminal prompt displaying the code

Exactly. That confirms the **loss lifecycle is working correctly**.

When your final guess was wrong:

- The game did **not** ask for another guess.
- It displayed the secret code.
- It returned to the normal terminal prompt.
- Therefore, the game correctly stops after the last allowed turn.

So we should **not count loss handling or the guess limit as a bug**.

## Current before-change findings

Requirement	Result
Duplicate-aware feedback	✗ Bug
No matches	✓ Works
Mixed exact/partial	✓ Works
Invalid guess length	✓ Works
Invalid symbols	✓ Works
Invalid guess doesn't consume turn	✓ Works
Loss after final turn	✓ Works

Requirement	Result
Secret shown after loss	✓ Works
Readable guess history	✗ Missing
Difficulty modes	✗ Missing
Final-turn win	🔍 Still need to test
State after winning	🔍 Still need to test
Quit with <code>q</code>	🔍 Still need to test

Let's **not change anything yet**.

### Next: test the winning lifecycle

We want to verify whether winning also cleanly ends the game.

The easiest way:

1. Run the game with **F5/debugger**.
2. Put a breakpoint at:

🔗 Python

```
exact, partial = score(self.code, raw)
```

3. When it pauses, look at:

🔗 Plain text

```
self.code
```

4. Enter that exact code as your guess.
5. You should see:

🔗 Plain text

```
Cracked the code!
```

6. Confirm it returns to the normal terminal prompt and **doesn't ask for another guess**.

Then tell me what happens. After that we'll test the **final-turn win** and `q`, and then we'll have our complete before-change bug/feature list.

it says cracked the code and returns back to the terminal prompt

Perfect. That confirms **win handling also works correctly**.

So we can mark these as verified working:

- ☒ Correct guess → "Cracked the code!"
- ☒ Game stops immediately after winning
- ☒ Returns to terminal prompt
- ☒ No further guesses are accepted

## We now have 2 confirmed problems

1. ☒ **Duplicate feedback bug**
  - Example: secret `5333`, guess `3333`
  - Current: `Exact: 3, Partial: 1`
  - Correct: `Exact: 3, Partial: 0`
2. ☒ **Readable guess history is missing**
  - `self.history` stores guesses internally.
  - But the player cannot see previous guesses/feedback during the game.
3. ☒ **Difficulty modes are completely missing**
  - No Easy/Medium/Hard selection.
  - Current game always uses 4 digits, symbols 1–6, and 10 turns.

## Still one small verification

We should test `q` / **quitting**, because the README specifically lists quitting as something to test.

Run the game and simply enter:

</> Plain text

q

Tell me what happens. Then we'll have the **complete before-change report**, and I'll help you decide exactly what to show in your 10-second before video.

after q the game ends midway and goes back to the terminal prompt

Good. That confirms **quit handling works too**.

## ✓ Complete before-change assessment

We can now separate the requirements into three groups:

### ✗ Confirmed bugs / missing features

#### 1. Duplicate-aware feedback is incorrect

- 5333 vs 3333 gives 3 exact, 1 partial instead of 3 exact, 0 partial.

#### 2. Guess history is not displayed

- Previous guesses and their feedback aren't shown to the player.

#### 3. Difficulty modes are missing

- No difficulty selection or different configurations.

## ✓ Already working

- Exact full match → win.
- No-match feedback.
- Mixed exact/partial feedback.
- Invalid length rejected.
- Invalid symbols rejected.
- Invalid guesses don't consume a turn.
- Final wrong guess ends the game and reveals the code.
- Winning ends the game immediately.
- q quits midway.
- Game doesn't continue after win/loss.

The README also specifically requires the feedback algorithm to work across the different difficulty configurations, so that will need to be verified **after** we add difficulty modes.



Lab\_4\_VibeCoding\_Student\_handout

## One important thing

**Do not modify the code yet.** We've now done the before-state investigation you wanted.

For your **before video**, the strongest evidence is:

1. Show the duplicate bug ( 5333 / 3333 → wrong partial).
2. Briefly show that previous guesses aren't displayed.
3. Show that there is no difficulty selection.

Those directly demonstrate the things we actually need to fix.

Once you're ready, our next step can be to create the **exact prompts for Task 1, Task 2, Task 3, and Task 4**, one at a time, so you can use them in your VibeCoding workflow and make a separate Git commit for each task.

just to be more accurate can you give me the trials to try to make a proper before video showing bugs

Yes. Since your goal is a **proper “before” video**, let's make the trials systematic so you don't accidentally record something that isn't actually a bug.

You **don't need to change any code**. Run the current version exactly as it is.

## Trial 1 — Duplicate-symbol bug ★ Most important

This is the clearest confirmed bug.

Use the debugger to find the secret code first.

For example, if:

</> Plain text

Secret = 5333

enter:

</> Plain text

3333

## Expected correct result

</> Plain text

Exact: 3

Partial: 0

## Current result

</> Plain text

Exact: 3

Partial: 1

📹 Record this one for your before video.

This directly demonstrates the duplicate-aware feedback requirement.

## Trial 2 — Another duplicate case

Try another case where duplicates can expose the problem.

For example:

</> Plain text

```
Secret = 1122
Guess = 1111
```

The correct result should be:

</> Plain text

```
Exact: 2
Partial: 0
```

The important thing is that the two **1** s in the secret can only be counted once each.

If you want to use the actual randomly generated code, inspect `self.code` in the debugger and construct a guess with repeated digits.

🧑‍🔧 You don't necessarily need this in the video if Trial 1 clearly demonstrates the bug.

## Trial 3 — No-match case

If the secret is:

</> Plain text

```
2252
```

try:

</> Plain text

```
1111
```

You should get:

</> Plain text

Exact: 0  
Partial: 0

✅ This is **working**, so don't present it as a bug.

This is useful if your instructor asks whether you tested the other feedback cases.

## Trial 4 — Mixed exact + partial

Using:

</> Plain text

Secret = 2252  
Guess = 2522

you should get:

</> Plain text

Exact: 2  
Partial: 2

✅ This is also working.

Again, don't present it as a bug.

## Trial 5 — History problem ★

Start a fresh game.

Make several valid guesses, for example:

</> Plain text

1234

then:

</> Plain text

2345

then:

</> Plain text

3456

After each guess, look at the terminal.

## What you should observe

You only see the feedback for the **current guess**.

The previous guesses aren't presented as a readable history such as:

</> Plain text

Guess	Exact	Partial
1234	1	1
2345	0	2
3456	2	0

📺 This is a good second thing to capture in the before video.

You can deliberately make 2–3 guesses quickly so it's obvious that the game isn't showing a history.

## Trial 6 — Invalid length

Enter:

</> Plain text

123

## Expected

The game should reject it and **not consume a turn**.

Then enter a valid guess.

✅ This is working.

Don't call this a bug.

## Trial 7 — Invalid symbols

Enter:

</> Plain text

9999

## Expected

The game should reject it because the current game only accepts digits **1-6**.

The turn should remain unchanged.

✅ Working.

## Trial 8 — Difficulty modes ★ Missing feature

Start a completely fresh game.

Look at the beginning of the game.

Currently, there is **no difficulty selection**, such as:

</> Plain text

Choose difficulty:

1. Easy
2. Medium
3. Hard

The current game simply starts with its fixed configuration.

🧩 This is another strong before-video item.

You don't actually need to play anything for this trial. Just show the game starting and demonstrate that there is no difficulty option.

## Trial 9 — Win

Use the debugger to see the secret, then enter it exactly.

For example:

</> Plain text

```
Secret = 2252
Guess = 2252
```

You should see:

```
</> Plain text
```

```
Cracked the code!
```

and the game ends.

✅ Working.

## Trial 10 — Loss

You already tested this.

Use up all available turns with incorrect guesses.

At the end:

```
</> Plain text
```

```
The code was XXXX
```

and the game ends.

✅ Working.

## Trial 11 — Quit

Enter:

```
</> Plain text
```

```
q
```

The game immediately ends and returns to the terminal.

You already tested this too.

✅ Working.

## What I recommend for your 10-second BEFORE video

Because the handout specifically asks for a **10-second before-changes video**, don't try to cram every test into it.  [Lab\\_4\\_VibeCoding\\_Student\\_handout](#)

I'd record this sequence:

### 0–3 seconds — Duplicate bug

Show:

 Plain text

Secret: 5333

Guess: 3333

Exact: 3   Partial: 1

The wrong **Partial: 1** is the key evidence.

### 3–6 seconds — History problem

Make two guesses and show that only the latest feedback is displayed rather than a guess history.

### 6–10 seconds — Missing difficulty

Restart/show the beginning of the game and show that it immediately starts with the fixed game instead of asking for a difficulty.

**That's much stronger than showing working features**, because your before video is supposed to establish what needs fixing.

## Your complete test checklist

You can keep this beside you:

 Plain text

BEFORE TESTING

[✓] Duplicate symbols	→ BUG
[✓] No matches	→ WORKS
[✓] Mixed exact/partial	→ WORKS
[✓] History	→ MISSING
[✓] Invalid length	→ WORKS
[✓] Invalid symbols	→ WORKS
[✓] Invalid doesn't use turn	→ WORKS
[✓] Win	→ WORKS
[✓] Loss	→ WORKS

[✓] Quit → WORKS  
[✓] Difficulty modes → MISSING

The important point is: **don't manufacture bugs for the video**. Showing the three genuine deficiencies—duplicate scoring, missing history, and missing difficulty—is enough to establish the before state accurately.

the secret code is 3 6 5 3

Great. Since your current secret is **3653**, we can use it to make the before-video tests more accurate.

### Trial 1 — Duplicate bug

Enter:

</> Plain text

3333

For secret **3653** :

- Position 1: **3** vs **3** → **exact**
- Position 2: **6** vs **3** → no
- Position 3: **5** vs **3** → no
- Position 4: **3** vs **3** → **exact**

So the correct result is:

</> Plain text

Exact: 2  
Partial: 0

But the current buggy code will likely report:

</> Plain text

Exact: 2  
Partial: 1

because it sees another **3** in the secret without properly consuming the already-matched occurrences.

Try **3333** now and tell me exactly what the game displays. This is probably the strongest bug to capture in your before video.

exact 2 partial 2

Yes — that confirms the duplicate bug very clearly. 👍

Your secret is:

</> Plain text

3653

Your guess:

</> Plain text

3333

The current game says:

</> Plain text

Exact: 2  
Partial: 2

But the correct result is:

</> Plain text

Exact: 2  
Partial: 0

Why?

The two 3 s that are already exact matches must be consumed.

Position	Secret	Guess	Result
1	3	3	Exact
2	6	3	—
3	5	3	—
4	3	3	Exact

Both **3** s in the secret have already been used as exact matches. There are **no remaining 3s** available to count as partial matches.

So this is an excellent **before-video bug demonstration**.

### For your video

Keep the game showing:

 Plain text

```
Secret: 3653
Guess: 3333
Exact: 2 Partial: 2
```

That gives you concrete evidence of the broken behavior.

**Don't fix anything yet.** We should continue with the remaining before tests using your actual secret **3653**, so your video/testing is accurate.

ill find a new secret key, give me all tests so that i can make a proper video

Absolutely. Let's do **one clean, systematic before-test session**. You can use a fresh secret code and work through these in order. **Do not change the code yet.**

Your goal is to identify what is genuinely broken/missing versus what already works.

## 1. First, get the new secret

Run the game in VS Code debugger and inspect:

 Plain text

```
self.code
```

Write it down as a 4-digit code, for example:

 Plain text

```
Secret = 3653
```

Don't tell me the code unless you want me to calculate the expected results for you.

## Tests for your BEFORE video

### Test A — Duplicate feedback ★ MUST SHOW

We want a secret containing a repeated digit.

For example, if your secret is:

</> Plain text

3653

use:

</> Plain text

3333

For 3653 / 3333, the **correct** result is:

</> Plain text

Exact: 2

Partial: 0

If the game gives a different partial count, you've demonstrated the duplicate bug.

### 🎥 Record this

Show:

</> Plain text

Guess: 3333

Exact: 2

Partial: 2

or whatever your program actually produces.

**Don't edit anything afterward.**

## Test B — No matches

Use a guess containing digits that aren't in the secret.

For example, if your secret contains only **3,6,5** , you could use:

</> Plain text

1111

For a secret **3653** :

</> Plain text

Exact: 0

Partial: 0

## Expected

The current program should correctly report:

</> Plain text

Exact: 0

Partial: 0

✓ This is a **working test**, not a bug.

You can mention it in your test record but don't present it as a failure.

## Test C — Mixed exact + partial

We want one guess containing both exact and partial matches.

For secret:

</> Plain text

3653

try:

</> Plain text

3561

Let's calculate it:

- 3 → exact
- 5 → exists in secret but wrong position → partial
- 6 → exists but wrong position → partial
- 1 → doesn't exist

Expected:

</> Plain text

Exact: 1

Partial: 2

✓ If your program gives that, this case works.

## Test D — Invalid length ★

Enter:

</> Plain text

123

The program should reject it.

Then check the remaining turns.

### Expected

The turn count should **not decrease**.

For example:

</> Plain text

Turns remaining: 7

before entering 123 , and still:

</> Plain text

Turns remaining: 7

after rejection.

✓ This currently works.

## Test E — Invalid symbols ★

Enter:

</> Plain text

9999

The current game only accepts digits `1-6`.

### Expected

It should reject the guess and **not consume a turn**.

✓ Currently works.

## Test F — Guess history ★ MUST SHOW

Make at least 3 valid guesses.

For example:

</> Plain text

1234

2345

3456

Look at the terminal after each guess.

The requirement is for a **readable guess history**, so ideally you would see something like:

</> Plain text

Guess	Exact	Partial
1234	1	1
2345	0	2
3456	2	0

### Current behavior

The game only shows the current guess's feedback.

The previous guesses aren't displayed as a readable history.

✖ Missing feature.

🎥 Record this

Make two or three guesses and capture the fact that the previous guesses aren't shown.

## Test G — Difficulty modes ★ MUST SHOW

Restart the game completely.

Watch what happens **before you make your first guess**.

The requirement is to have difficulty modes that change:

- code length
- symbol range
- number of allowed guesses

The current game should simply start with its fixed configuration.

There is no:

⌕ Plain text

Choose difficulty:

1. Easy
2. Medium
3. Hard

✖ Missing feature.

🎥 Record this

Show the game starting without any difficulty selection.

This is a strong before-video demonstration.

## Test H — Win

Use the debugger to inspect the secret.

Then enter the exact code.

Example:

</> Plain text

Secret: 3653

Guess: 3653

Expected:

</> Plain text

Cracked the code!

and the game ends.

✅ Working.

You don't need to show this as a bug.

## Test I — Loss

Start another game.

Use incorrect valid guesses until you reach the last turn.

On the final wrong guess, the game should:

1. Stop accepting guesses.
2. Display the secret.
3. Return to the terminal.

✅ Working.

You've already verified this.

## Test J — Quit

Start another game and enter:

</> Plain text

q

Expected:

- Game ends immediately.
- Returns to terminal.
- No additional guess is requested.

✔ Working.

## Test K — Final-turn win

This one is worth testing because the README specifically calls out **final-turn wins**.

Start a fresh game.

Use incorrect guesses until you have **1 turn remaining**.

Then inspect the secret with the debugger and enter the correct code.

### Expected

Even though it's the final allowed turn:

</> Plain text

Cracked the code!

The game should end as a **win**, not a loss.

If that works, mark it:

✔ Final-turn win works.

## Your complete checklist

Use this while testing:

Test	What you're checking	Result
A	Duplicate symbols	✗ Bug
B	No matches	✔ Working
C	Mixed exact/partial	✔ Working
D	Invalid length	✔ Working

Test	What you're checking	Result
E	Invalid symbols	✓ Working
F	Readable history	✗ Missing
G	Difficulty modes	✗ Missing
H	Normal win	✓ Working
I	Normal loss	✓ Working
J	Quit	✓ Working
K	Final-turn win	🔍 Test

## 🎬 For the actual 10-second BEFORE video

Don't try to cram all 11 tests into 10 seconds. Focus on the **three actual deficiencies**:

### 0–4 sec — Duplicate bug

⌄ Plain text

Secret: XXXX

Guess: XXXX

Exact: X

Partial: X ← incorrect

### 4–7 sec — History

Make two quick guesses and show that there is no previous-guess history.

### 7–10 sec — Difficulty

Restart and show that the game immediately starts without a difficulty-selection option.

That gives you a clean, defensible **before** video showing the actual things you're going to fix, rather than pretending the already-working features are broken.

The lab specifically asks you to run the game, record the before state, then fix the broken code and add the listed features.

 Lab\_4\_VibeCoding\_Student\_handout

5153 is the secret code

Perfect. Your current secret is **5153**. We can now calculate the expected results for your tests exactly.

## Test 1 — Duplicate bug ★

Enter:

</> Plain text

3333

For:

</> Plain text

Secret: 5153

Guess: 3333

Correct result:

- Position 3 → 5 vs 3 → no
- Position 4 → 3 vs 3 → exact
- There is only **one** 3 in the secret, and it has already been used as an exact match.

So the correct answer is:

</> Plain text

Exact: 1

Partial: 0

Your current program should incorrectly count the same 3 again as partial.

Run 3333 and tell me exactly what it displays. That will be our strongest confirmed bug for the before video.

give all other tests too

Sure. With secret = 5153, here are the exact remaining tests. Don't change the code yet.

## 2. No-match test

Enter:

</> Plain text

2466

Secret:

</> Plain text

5153

None of 2,4,6 appears in the secret.

Expected:

</> Plain text

Exact: 0

Partial: 0

✓ This should work.

### 3. Mixed exact + partial

Enter:

</> Plain text

1536

Compare:

</> Plain text

Secret: 5153

Guess: 1536

- 1 → position 1 vs 5 → partial
- 5 → position 2 vs 1 → partial
- 3 → position 3 vs 5 → no
- 6 → not in secret → no

Expected:

</> Plain text

Exact: 0

Partial: 2

This is a good test of normal partial matching.

## 4. Another mixed test

Enter:

</> Plain text

5135

Compare:

</> Plain text

Secret: 5153

Guess: 5135

- 5 → exact
- 1 → exact
- 3 → partial
- 5 → partial

But there are only **two 5 s in the secret**, and both are already used as the exact 5 matches? Actually, the secret is 5 1 5 3 ; the guess is 5 1 3 5 .

So:

- position 1: 5 exact
- position 2: 1 exact
- remaining secret: 5,3
- remaining guess: 3,5
- both are partial

Expected:

</> Plain text

Exact: 2

Partial: 2

This case should work with the current implementation.

## 5. Invalid length

Enter:

</> Plain text

123

**Expected:** rejection message.

Before entering it, note your remaining turns. After entering it, the number of turns should be **unchanged**.

☒ Working behavior.

Then try:

</> Plain text

12345

Again, it should reject it without consuming a turn.

## 6. Invalid symbols

Enter:

</> Plain text

9999

The current game only accepts **1-6**.

**Expected:** rejection and **no turn consumed**.

Also try:

</> Plain text

0000

Same expected behavior.

## 7. History test ★

Start a **new game** because your current game will accumulate the guesses above.

Make three valid guesses, for example:

</> Plain text

```
1234
2345
3456
```

After the third guess, look at the screen.

## What we're looking for

The game should ideally show something like:

</> Plain text

Guess	Exact	Partial
1234	...	
2345	...	
3456	...	

But the current implementation does **not** display the previous guesses.

✗ **Missing feature — record this for your before video.**

## 8. Difficulty test ★

Start another fresh game.

Look at the very beginning.

The current program should immediately begin the fixed game instead of asking:

</> Plain text

```
Choose difficulty
```

There are currently no Easy/Medium/Hard modes.

✗ **Missing feature — record this for your before video.**

The assignment specifically requires difficulty modes that vary **code length, symbol range, and allowed guesses**.



Lab\_4\_VibeCoding\_Student\_handout

## 9. Normal win

For your current secret:

</> Plain text

5153

Enter:

</> Plain text

5153

Expected:

</> Plain text

Cracked the code!

Then it should return to the terminal.



Working.

## 10. Quit

Start a new game and enter:

</> Plain text

q

**Expected:** game ends immediately and returns to the terminal.

You already verified this.



Working.

## 11. Loss

Start a fresh game and deliberately use wrong valid guesses until the final turn.

**Expected:**

</> Plain text

The code was XXXX

Then the game ends.

You already verified this.

✅ Working.

## 12. Final-turn win

This is the last special case to verify.

Start a new game and deliberately make incorrect guesses until **1 turn remains**.

Then use the debugger to inspect the secret and enter the exact code.

For example, if the secret is still `5153` :

</> Plain text

5153

**Expected:**

</> Plain text

Cracked the code!

even though it is the last turn.

## Your final BEFORE checklist

</> Plain text

- |                             |   |   |         |
|-----------------------------|---|---|---------|
| 1. Duplicate symbols        | → | ✗ | BUG     |
| 2. No matches               | → | ✓ |         |
| 3. Mixed exact/partial      | → | ✓ |         |
| 4. Invalid length           | → | ✓ |         |
| 5. Invalid symbols          | → | ✓ |         |
| 6. Invalid doesn't use turn | → | ✓ |         |
| 7. Readable history         | → | ✗ | MISSING |
| 8. Difficulty modes         | → | ✗ | MISSING |
| 9. Normal win               | → | ✓ |         |
| 10. Quit                    | → | ✓ |         |
| 11. Loss                    | → | ✓ |         |
| 12. Final-turn win          | → | 🔍 | VERIFY  |

For the **10-second before video**, prioritize **#1 duplicate bug**, **#7 history**, and **#8 missing difficulty modes**. Those are the three clearest things that actually need to be changed.

ill show only the bugs part

Yes — that's the best approach for a **10-second before video**.

Show only the **three confirmed deficiencies**:

### 1. Duplicate feedback bug

- Secret: 5153
- Guess: 3333
- Current result should incorrectly show:

</> Plain text

Exact: 1  
Partial: 1

Correct should be:

</> Plain text

Exact: 1  
Partial: 0

### 2. Missing guess history

- Make 2–3 guesses.
- Show that the game only displays the latest feedback instead of a readable history.

### 3. Missing difficulty modes

- Restart the game.
- Show that there is no Easy/Medium/Hard selection and the game starts directly.

**Don't spend video time showing the things that already work.** The three above are enough to establish the before state and align with the assignment's requirement to fix the broken code and add the README features.  Lab\_4\_VibeCoding\_Student\_handout

Once you've recorded it, **don't modify the code yet**. We can then start Task 1 and make the first separate commit.

its evident that we dont have easy medium selection.. i dont need to show it now for before as ill be showing it in after video

Exactly. 👍 You don't need to demonstrate the missing difficulty selection in the **before video** because its absence is already obvious, and the **after video** can clearly show the new Easy/Medium/Hard selection.

For the before video, I'd keep it focused on the **actual observable bug**:

## 🎬 Before video

### 1. Duplicate feedback bug

⌕ Plain text

Secret: 5153

Guess: 3333

Result: Exact: 1, Partial: 1 ❌

### 2. Missing history

Make a couple of guesses and show that previous guesses aren't displayed.

That's enough. It keeps the video clean and directly establishes what you're going to improve.

Then in the **after video**, you can demonstrate:

- Easy / Medium / Hard selection
- Correct duplicate handling
- Readable history
- The improved game behavior

So yes — **skip the difficulty demonstration in the before video**.

i want to try with this secret key--> 3435

Perfect. With secret **3435**, here are the best tests for your before-state.

### 1. Duplicate bug ★

Enter:

</> Plain text

3333

Compare:

</> Plain text

Secret: 3435

Guess: 3333

Correct result:

- Position 1 → exact
- Position 2 → 4 vs 3 → no
- Position 3 → exact
- Position 4 → 5 vs 3 → no

So:

</> Plain text

Exact: 2

Partial: 0

The current buggy logic may count the 3 s incorrectly and produce an extra partial.

**This is the one I recommend recording.**

## 2. History bug

After that, make a couple more valid guesses, for example:

</> Plain text

1234

4561

The game will show feedback for each current guess, but it won't present a readable list of your previous guesses.

**This demonstrates the missing history feature.**

**For your before video**

I'd show:

**First:** 3435 + 3333 → incorrect duplicate feedback.

**Then:** make 2 guesses → show that there is no visible guess history.

That's all you need. No need to show the difficulty absence since you'll demonstrate the new difficulty selection in the after video.

i have shown this in my video Mastermind — enter four digits from 1 to 6.

10 turns left > 3333

Exact: 2 Partial: 2

9 turns left > 1234

Exact: 1 Partial: 1

8 turns left > 4561

Exact: 0 Partial: 2

7 turns left > -- do i have to show all 10 turns?

No, **you do not have to show all 10 turns**. In fact, for a 10-second before video, you shouldn't.

What you've shown is already enough to demonstrate the two important before-state issues:

#### </> Plain text

Mastermind — enter four digits from 1 to 6.

10 turns left > 3333

Exact: 2 Partial: 2 ✗ duplicate-counting bug

9 turns left > 1234

Exact: 1 Partial: 1

8 turns left > 4561

Exact: 0 Partial: 2

7 turns left >

## One thing to check

Your video currently shows the **duplicate bug very clearly** with 3333, because for secret 3435 the correct result should be:

#### </> Plain text

Exact: 2

Partial: 0

but the game shows:

</> Plain text

Exact: 2

Partial: 2

That's excellent evidence.

And your next guesses demonstrate that the game continues while consuming turns, but **you don't need to play through all 10 turns** just to prove the bug.

For the **history issue**, your sequence of three guesses also helps: the game only shows each current result rather than a visible accumulated history.

**So I'd stop the before video here.**

You can leave it at:

</> Plain text

7 turns left >

You **do not need to intentionally lose the game** or use all 10 turns.

The lab says to record a **10-second video before changes**, not to demonstrate every test case in the video.



Lab\_4\_VibeCoding\_Student\_handout

Your before video is therefore good enough. **Don't change the code yet.** Next we can start fixing the first task with a proper VibeCoding prompt.

okay lets begin with task 1: i guess we need to implement feedback so that each code position can be counted at most once and solve the exact matches before partial matches

Yes. **Task 1 is exactly that.** The goal is to fix `logic.py` so feedback is calculated in two stages:

1. **Count exact matches first.**
2. Remove/consume those matched positions.
3. **Then count partial matches** only from the remaining unmatched code and guess symbols.
4. Each code position/symbol occurrence can be counted **at most once**.

Your current logic:

</> Python

```
exact = sum(a == b for a, b in zip(code, guess))
```

```
partial = sum(ch in code for ch in guess) - exact
```

doesn't consume matched occurrences, which caused your `3435` / `3333` test to produce `Exact: 2`, `Partial: 2`.

## VibeCoding prompt for Task 1

Since your lab asks you to use prompts and make separate commits for each task, I'd use a focused prompt rather than asking it to change the whole project:

Task 1 prompt

Fix Task 1 in the Mastermind game.

The current feedback logic in `logic.py` incorrectly counts duplicate symbols. Each occurrence in the secret code and each occurrence in the guess must be counted at most once.

**Important:** Don't commit yet until we test it. After your VibeCoding tool makes the change, we'll inspect `logic.py`, run the tests, and use the debugger if necessary. Then, once Task 1 is verified, we'll make the **Task 1 Git commit** separately.

lets continue task 1...i have given the prompt earlier of identifying all exact postion matches, these not being counted again and comparing only the remaining unmatched codes. lets proceed with this task.. im not sure what code to modify? is it logic.py???

Yes — `logic.py` is the correct file for Task 1. 👍

From the code we inspected earlier, the bug is specifically in the scoring function there. `game.py` calls the scoring function, but it doesn't contain the duplicate-counting logic itself.

## What we're changing

In `logic.py`, you currently have logic equivalent to:

</> Python

```
exact = sum(a == b for a, b in zip(code, guess))
partial = sum(ch in code for ch in guess) - exact
```

The problem is this part:

</> Python

```
ch in code
```

It only asks "does this character exist anywhere in the code?" It does not keep track of whether that occurrence has already been used for an exact match or another partial match.

## Task 1 implementation approach

We want `logic.py` to do:

### </> Plain text

Step 1 → Find exact matches

↓

Mark those positions as used

↓

Step 2 → Look only at remaining positions

↓

Find partial matches

↓

Return exact, partial

For your before-test:

### </> Plain text

Secret: 3435

Guess: 3333

the algorithm should do:

### </> Plain text

3 → exact ✓

4 → unmatched

3 → exact ✓

5 → unmatched

After consuming those two exact `3`s, there are **no remaining** `3`s in the secret.

Therefore:

### </> Plain text

Exact: 2

Partial: 0

## What I recommend doing now

Since you're using VibeCoding, **don't manually write the implementation yet.**

Open `logic.py` in VS Code and give your VibeCoding tool the prompt you already prepared:

Identify all exact position matches first. These exact matches must not be counted again. Then compare only the remaining unmatched code positions and guess positions to calculate partial matches. Each code position can be counted at most once.

Tell it specifically:

**Modify only `logic.py` for Task 1. Do not modify `game.py` or implement any other tasks yet.**

After it makes the change, **show me the new `logic.py` code before we move on.** I'll check whether the implementation actually satisfies the duplicate rules, and then we'll test it with `3435 / 3333` and the other edge cases.

ill use you as my vibe coding tool.. lets go ahead with this task

Absolutely. We'll do **Task 1 only** and leave `game.py` and the other tasks untouched.

## Task 1 goal

Modify `logic.py` so that:

1. Exact matches are counted first.
2. Exact-match positions are consumed.
3. Only the remaining unmatched positions are checked for partial matches.
4. No code/guess occurrence can be counted more than once.

I'm ready to make that change.

ok

Great. Let's implement **Task 1** in `logic.py` only.

Please open `logic.py` in VS Code and **replace its current scoring logic with this:**

`</>` Python

```
def score(code, guess):
 exact = 0
 remaining_code = []
 remaining_guess = []

 # First: find exact matches
 for code_digit, guess_digit in zip(code, guess):
 if code_digit == guess_digit:
 exact += 1
 else:
```

```
remaining_code.append(code_digit)
remaining_guess.append(guess_digit)

Second: find partial matches among unmatched positions
partial = 0
for digit in remaining_guess:
 if digit in remaining_code:
 partial += 1
 remaining_code.remove(digit)
```

## Why this fixes the bug

For:

</> Plain text

Secret: 3435

Guess: 3333

the first pass finds **2 exact matches** and removes those positions from consideration.

The remaining secret is effectively:

</> Plain text

45

and the remaining guess is:

</> Plain text

33

There are no **3** s left in the unmatched secret, so:

</> Plain text

Exact: 2

Partial: 0

This also ensures that a repeated symbol can only be counted as many times as it actually occurs in the **remaining** secret.

## Important

For now, only change **logic.py**. Don't touch:

- `game.py`
- `main.py`
- `README.md`
- `requirements.txt`

Once you've made this change, **don't commit yet**.

Run the game and test:

</> Plain text

3435 / 3333

Tell me what result you get. Then we'll test the other duplicate cases before making the **Task 1 commit**.

i have fixed the broken part of the code wherever necessary

Great. Before we move to Task 2, let's **verify Task 1 properly**. Don't make any other changes yet.

### Test the fixed feedback with these cases

If your secret is still `3435`, run:

Secret	Guess	Expected
<code>3435</code>	<code>3333</code>	Exact 2, Partial 0
<code>3435</code>	<code>1234</code>	Exact 1, Partial 1
<code>3435</code>	<code>4561</code>	Exact 0, Partial 2
<code>3435</code>	<code>3435</code>	Exact 4, Partial 0

The first one is the **critical regression test** because it was the bug shown in your before video.

### Also test a stronger duplicate case

Use the debugger or temporarily use a known secret:

</> Plain text

Secret: 5333

Guess: 3333

Expected:

</> Plain text

Exact: 3

Partial: 0

This confirms that the same 3 occurrences aren't being counted again.

## After testing

If all of those produce the expected results, **Task 1 is verified**.

Then we'll:

1. Review exactly what changed in `logic.py`.
2. Run the tests one more time.
3. Create a **separate Git commit for Task 1**, as required by your lab.
4. Only then move to **Task 2**.

Don't start Task 2 until we've confirmed Task 1.

error has occurred in game.py where we are trying to import feedback from logic

Yes — this is likely because the existing `game.py` expects a function named `feedback`, while the code we just put into `logic.py` used the name `score`.

We need to preserve the existing interface. **Don't change `game.py` yet.** Let's inspect the exact current `logic.py` and the import line in `game.py` so we fix this without introducing another problem.

Please paste these two small pieces here:

1. The **first 5–10 lines of `game.py`**, especially the import.
2. The **entire current `logic.py`**.

Then I'll give you the exact corrected `logic.py`.

ok

Great. Please paste those two pieces here:

1. `game.py` — **first 5–10 lines**, especially the import line.
2. `logic.py` — **the entire file as it looks now**.

I'll then correct the naming/interface issue while keeping **Task 1 only** and without changing `game.py` unnecessarily.

```
import random
from logic import feedback

class Mastermind:
 def __init__(self):
 self.code = [str(random.randint(1, 6)) for _ in range(4)]
 self.history = []
 self.turns = 10

 def run(self):
 print("Mastermind — enter four digits from 1 to 6.")
 while self.turns:
 raw = input(f"{self.turns} turns left > ").strip()
 if raw.lower() == "q":
 return
 if len(raw) != 4 or any(ch not in "123456" for ch in raw):
 print("Enter exactly four digits from 1 to 6.")
 continue
 guess = list(raw)
 exact, partial = feedback(self.code, guess)
 self.history.append((raw, exact, partial))
 self.turns -= 1
 print("Exact:", exact, " Partial:", partial)
 if exact == 4:
 print("Cracked the code!")
 return
 print("The code was", "".join(self.code))
```

Perfect — this confirms the issue. `game.py` imports:

`</>` Python

```
from logic import feedback
```

and then calls:

`</>` Python

```
feedback(self.code, guess)
```

So we must keep the function name `feedback`. My earlier example incorrectly changed it to `score`; that's why you got the import error.

### Fix `logic.py`

Keep the function name as `feedback` and use the duplicate-aware logic:

</> Python

```
def feedback(code, guess):
 exact = 0
 remaining_code = []
 remaining_guess = []

 # First: count exact matches
 for code_digit, guess_digit in zip(code, guess):
 if code_digit == guess_digit:
 exact += 1
 else:
 remaining_code.append(code_digit)
 remaining_guess.append(guess_digit)

 # Second: count partial matches only among unmatched positions
 partial = 0
 for digit in remaining_guess:
 if digit in remaining_code:
 partial += 1
 remaining_code.remove(digit)
```

Do not change `game.py`. Its existing interface is correct.

### Now test Task 1

With your secret:

</> Plain text

3435

enter:

</> Plain text

3333

You should now get:

</> Plain text

Exact: 2 Partial: 0

Then test:

</> Plain text

1234

Expected:

</> Plain text

Exact: 1 Partial: 1

And:

</> Plain text

4561

Expected:

</> Plain text

Exact: 0 Partial: 2

If those pass, we'll test the important repeated-code case and then make your **Task 1 Git commit**.

this is the issue: **xception has occurred: ImportError**

- 
- 

cannot import name 'feedback' from 'logic' (c:\Users\Preeti Kalburgi\Desktop\SE LAB 4\20\_mastermind\logic.py)

File "C:\Users\Preeti Kalburgi\Desktop\SE LAB 4\20\_mastermind\game.py", line 2, in <module> from logic import feedback File "C:\Users\Preeti Kalburgi\Desktop\SE LAB 4\20\_mastermind\main.py", line 1, in <module> from game import Mastermind ImportError: cannot import name 'feedback'

```
from 'logic' (c:\Users\Preeti Kalburgi\Desktop\SE LAB
4\20_mastermind\logic.py)
```

Yes — the traceback confirms the problem is specifically in `logic.py`:

</> Plain text

```
ImportError: cannot import name 'feedback' from 'logic'
```

`game.py` is doing:

</> Python

```
from logic import feedback
```

but Python cannot find a function called `feedback` inside your current `logic.py`.

## Fix this first

Open `logic.py` and make sure the function is named **exactly**:

</> Python

```
def feedback(code, guess):
 exact = 0
 remaining_code = []
 remaining_guess = []

 # First, count exact matches
 for code_digit, guess_digit in zip(code, guess):
 if code_digit == guess_digit:
 exact += 1
 else:
 remaining_code.append(code_digit)
 remaining_guess.append(guess_digit)

 # Then, count partial matches from remaining positions
 partial = 0
 for digit in remaining_guess:
 if digit in remaining_code:
 partial += 1
 remaining_code.remove(digit)
```

**Important:** The first line must be:

</> Python

```
def feedback(code, guess):
```

—not `def score(...)`.

And make sure the indentation is exactly as shown.

## Then save both files

Press:

</> Plain text

Ctrl + S

You do not need to change `game.py`.

Your existing `game.py` is already expecting:

</> Python

```
from logic import feedback
```

and:

</> Python

```
exact, partial = feedback(self.code, guess)
```

So keeping `feedback` preserves the existing interface.

## Then run again

Press **F5** in VS Code.

If the ImportError disappears, test your original bug:

</> Plain text

Secret: 3435

Guess: 3333

Expected:

</> Plain text

Exact: 2   Partial: 0

If you still get the ImportError after making sure the function is named `feedback`, **paste your current `logic.py` here exactly as it appears**. I'll identify the issue before we make any further changes.

ill open notebook to save changes to logic.py

Yes. Open your `logic.py` **notebook/editor**, make sure the function is named `feedback`, and save it.

Use this exact version:

</> Python

```
def feedback(code, guess):
 exact = 0
 remaining_code = []
 remaining_guess = []

 # First, count exact matches
 for code_digit, guess_digit in zip(code, guess):
 if code_digit == guess_digit:
 exact += 1
 else:
 remaining_code.append(code_digit)
 remaining_guess.append(guess_digit)

 # Then, count partial matches from remaining positions
 partial = 0
 for digit in remaining_guess:
 if digit in remaining_code:
 partial += 1
 remaining_code.remove(digit)
```

After saving:

1. Go back to `game.py`.
2. Press F5.
3. Use your test case:

</> Plain text

Secret: 3435  
Guess: 3333

4. Confirm you get:

&lt;/&gt; Plain text

Exact: 2 Partial: 0

**Don't make any other changes yet.** Once this runs successfully, we'll verify Task 1 with the other edge cases before committing it.

how do i open that im confused

No problem — I made that sound more complicated than it is. You **don't need to open a notebook**. You just need to open the `logic.py` file in VS Code.

## Do this step by step

1. Look at the **left side of VS Code** for the Explorer panel.
2. You should see your project files:

&lt;/&gt; Plain text

```
20_mastermind
├─ README.md
├─ game.py
├─ logic.py
├─ main.py
└─ requirements.txt
```

3. Click `logic.py`.
4. Its code will open in the main editor.
5. Select everything in `logic.py` with:  
**Ctrl + A**
6. Replace it with:

&lt;/&gt; Python

```
def feedback(code, guess):
 exact = 0
 remaining_code = []
 remaining_guess = []

 # First, count exact matches
 for code_digit, guess_digit in zip(code, guess):
 if code_digit == guess_digit:
 exact += 1
 else:
 remaining_code.append(code_digit)
 remaining_guess.append(guess_digit)

 # Then, count partial matches from remaining positions
```

```
partial = 0
for digit in remaining_guess:
 if digit in remaining_code:
 partial += 1
 remaining_code.remove(digit)
```

7. Press **Ctrl + S** to save.

## Then run it

Press **F5**.

If the game starts normally, test:

</> Plain text

3333

with your secret **3435** .

We want:

</> Plain text

Exact: 2 Partial: 0

Don't worry about the word "notebook" I used earlier — just use the normal **logic.py** file in your VS Code project.

no i mean i want to open notebook

Ah, got it — you mean **Jupyter Notebook**, not the **logic.py** file. 👍

If you want to open a Jupyter Notebook in VS Code:

## Option 1 — From VS Code

1. In VS Code, press:  
**Ctrl + Shift + P**
2. Type:

</> Plain text

Jupyter: Create New Blank Notebook

3. Select it.

4. A new `.ipynb` notebook will open.

## Option 2 — From the terminal

In your project terminal, run:

```
</> powershell
```

```
jupyter notebook
```

If Jupyter is installed, your browser should open the Jupyter Notebook interface.

If it says:

```
</> Plain text
```

```
jupyter is not recognized
```

tell me that message and I'll help you install/open it.

**One important distinction:** your actual Task 1 code should still be saved in `logic.py`. A Jupyter notebook would just be a separate place to experiment/test the scoring logic; it won't automatically update `logic.py`.

notepad im sorry

Ahh 😊 **Notepad**, got it!

If you want to open `logic.py` in regular Windows Notepad:

## From VS Code

1. In the left **Explorer**, right-click `logic.py`.
2. Choose **Reveal in File Explorer**.
3. Windows Explorer will open the folder.
4. Right-click `logic.py`.
5. Select **Open with** → **Notepad**.

You can then edit the file in Notepad and press **Ctrl + S** to save.

However, since you're already working in **VS Code**, I'd recommend editing `logic.py` directly in VS Code. It's much easier to see indentation and catch Python errors.

If you specifically want to use Notepad, that's completely fine too.

worked, 5662 is the secret key

Excellent! 🎉 That means the import problem is fixed and the game is running again.

Your new secret is **5662**. Let's verify **Task 1** before we commit anything.

## Test 1 — Duplicate bug

Enter:

</> Plain text

5666

Compare:

</> Plain text

Secret: 5662

Guess: 5666

Expected:

- Position 1: 5 = 5 → exact
- Position 2: 6 = 6 → exact
- Position 3: 6 = 6 → exact
- Position 4: 2 ≠ 6 → no match

So:

</> Plain text

Exact: 3

Partial: 0

## Test 2 — Complete match

Enter:

</> Plain text

5662

Expected:

</> Plain text

Exact: 4  
Partial: 0  
Cracked the code!

### Test 3 — No matches

For 5662 , try:

</> Plain text

1111

Expected:

</> Plain text

Exact: 0  
Partial: 0

### Test 4 — Partial matching

Try:

</> Plain text

6251

For 5662 / 6251 :

- 6 → partial
- 2 → partial
- 5 → partial
- 1 → no match

Expected:

</> Plain text

Exact: 0  
Partial: 3

### Most important

Start with 5666 and tell me what you get.

If you get:

</> Plain text

Exact: 3 Partial: 0

then our duplicate fix is behaving correctly for this case, and we'll continue with the remaining Task 1 tests.

Mastermind — enter four digits from 1 to 6.

10 turns left > 5666

Exact: 3 Partial: 0

9 turns left > 5662

Exact: 4 Partial: 0

Cracked the code!

PS C:\Users\Preeti Kalburgi\Desktop\SE LAB 4\20\_mastermind>  
this is my output

Perfect! 🎉 Task 1 is working for the first two critical tests.

Your output:

</> Plain text

5666

Exact: 3 Partial: 0

is exactly correct. This proves the duplicate issue we saw before has been fixed for this case.

And:

</> Plain text

5662

Exact: 4 Partial: 0

Cracked the code!

also confirms the normal winning case still works.

## Before we commit Task 1, do these 3 quick tests

We want to make sure the new logic didn't break other feedback cases.

### Test 3 — No matches

Start a new game and, if the secret is still 5662 , enter:

</> Plain text

1111

Expected:

</> Plain text

Exact: 0 Partial: 0

#### Test 4 — Partial matches

If the secret is 5662 , enter:

</> Plain text

6251

Expected:

</> Plain text

Exact: 0 Partial: 3

#### Test 5 — Mixed exact + partial

For secret 5662 , enter:

</> Plain text

6526

Expected:

- 6 → exact
- 5 → partial
- 2 → partial
- 6 → partial

So:

</> Plain text

Exact: 1 Partial: 3

Once these pass, **Task 1 is fully verified**. Then we'll make the separate Git commit before touching Task 2.

it is fully verified

Excellent.  **Task 1 is fully verified.**

Before moving to Task 2, let's create the required **separate Git commit for Task 1**. This keeps your lab history clean.

## Step 1 — Check what changed

In the VS Code terminal, run:

```
</> powershell
```

```
git status
```

You should see `logic.py` listed as modified.

## Step 2 — Review the change

Run:

```
</> powershell
```

```
git diff logic.py
```

You should see the old duplicate-unaware feedback replaced by the new exact-first/remaining-match logic.

## Step 3 — Commit Task 1

Run these commands **one at a time**:

```
</> powershell
```

```
git add logic.py
```

then:

```
</> powershell
```

```
git commit -m "Fix duplicate-aware feedback"
```

Then verify:

```
</> powershell
```

```
git status
```

Ideally it should say your working tree is clean.

## Don't push yet if you're unsure

After the commit, **stop there and show me the output of:**

```
</> powershell
```

```
git status
```

and

```
</> powershell
```

```
git log -1 --oneline
```

I'll confirm that Task 1 is safely committed before we start **Task 2: complete game lifecycle/history behavior.**

okay now for task 2, we need to complete the game cycle by adding clear win/loss handling, preserving guess history, enforcing the guess limit, and preventing state changes after the game ends.. you are my vibing tool so help me fix broken code

Your current `game.py` already has some of Task 2, but it has two important weaknesses:

- `self.history` is stored but **never shown to the player**.
- The lifecycle is handled through `return`, but we should make the **game-ended state explicit** so no further guesses can change state.

For this task, use this as your VibeCoding prompt:

Writing



Update `game.py` to complete the Mastermind game lifecycle.

Requirements:

1. Keep the existing `feedback(self.code, guess)` interface unchanged.
2. Preserve every valid guess and its Exact/Partial result in `self.history`.

3. Display a readable guess history to the player after each valid guess.
4. Handle a win clearly when the player guesses the entire code.
5. Handle a loss clearly when the player uses the final allowed turn, including revealing the code.
6. Enforce the 10-turn guess limit. Do not allow any guess after the limit is reached.
7. Invalid guesses must not consume a turn or modify history.
8. Quitting with q must end the game immediately.
9. After the game ends by win, loss, or quit, no further input or state changes should occur.
10. Keep the existing 10-turn limit and current four-digit 1–6 configuration; do not implement difficulty modes yet.
11. Modify only `game.py`. Do not change `logic.py`.

Use a clear game-ended state if needed so the lifecycle cannot continue after the game has ended.

## If you're letting your VibeCoding tool generate the change

Paste that prompt into your coding tool. Do not accept changes to `logic.py` for Task 2.

The resulting `game.py` should have the lifecycle roughly like this:

</> Python

```
import random
from logic import feedback

class Mastermind:
 def __init__(self):
 self.code = [str(random.randint(1, 6)) for _ in range(4)]
 self.history = []
 self.turns = 10
 self.game_over = False

 def run(self):
 print("Mastermind – enter four digits from 1 to 6.")

 while self.turns > 0 and not self.game_over:
 raw = input(f"{self.turns} turns left > ").strip()

 if raw.lower() == "q":
 self.game_over = True
 print("Game quit.")
```

## Important

This is **Task 2 only**.

Don't add:

- difficulty levels
- different code lengths
- different symbol ranges
- persistence
- CSV/JSON
- changes to `logic.py`

## Now verify it

After your VibeCoding tool makes the change, run:

```
</> powershell
```

```
python main.py
```

Test these **one at a time**:

### 1. Valid guesses/history

Make 2–3 valid guesses and confirm something like:

```
</> Plain text
```

```
Exact: 1 Partial: 1
```

```
Guess history:
```

```
1234 -> Exact: 1, Partial: 1
```

Then make another guess and confirm **both guesses remain** in history.

### 2. Invalid input

Try:

```
</> Plain text
```

```
123
```

and:

```
</> Plain text
```

```
7777
```

The turn count must **not decrease**, and these guesses must **not appear in history**.

### 3. Win

Enter the secret code.

Expected:

</> Plain text

Cracked the code!

and the program must return to PowerShell. It must not ask for another guess.

### 4. Loss

Use incorrect valid guesses until the 10th turn.

Expected:

</> Plain text

Out of turns!

The code was XXXX

and the program must terminate.

### 5. Quit

Start another game and enter:

</> Plain text

q

Expected:

</> Plain text

Game quit.

No further input should be requested.

Once those five checks pass, **don't commit yet**. Show me the resulting `game.py` or the test output, and I'll verify Task 2 before we make its separate Git commit.

i tried all the tests specified, history is maintained, invalid input is being handled carefully, cracked the code message on entering the same as secret key, loss for incorrect guesses and quit also works properly..

Perfect.  **Task 2 is fully verified.**

You have confirmed all four required areas:

-  **Guess history** is preserved and displayed.
-  **Invalid input** does not consume a turn or corrupt history.
-  **Win handling** displays `Cracked the code!` and ends the game.
-  **Loss handling** occurs after the allowed guesses and reveals the secret.
-  **Quit handling** ends the game cleanly.
-  **Guess limit** is enforced.
-  **No continued input after game ends.**

Now let's make the required **separate Task 2 commit**.

## 1. Check the changed files

```
</> powershell
```

```
git status
```

You should see `game.py` modified.

## 2. Review the Task 2 changes

```
</> powershell
```

```
git diff game.py
```

Make sure the changes are only related to the game lifecycle/history.

## 3. Commit Task 2

```
</> powershell
```

```
git add game.py
```

Then:

```
</> powershell
git commit -m "Complete Mastermind game lifecycle"
```

#### 4. Verify the commit

```
</> powershell
git status
```

Then:

```
</> powershell
git log --oneline -2
```

You should have **two separate commits**, approximately:

```
</> Plain text

Complete Mastermind game lifecycle
Fix duplicate-aware feedback
```

Don't push yet. Once you confirm the two commits are there, we'll move to **Task 3: difficulty modes** and implement them separately.

i want to rename my commit

Yes. Since the Task 2 commit is your **most recent commit**, you can rename it safely.

#### Rename the latest commit

Run:

```
</> powershell
git commit --amend -m "Task 2 - Complete game lifecycle"
```

Then verify:

```
</> powershell
git log --oneline -2
```

You should see something like:

**</> Plain text**

```
xxxxxxx Task 2 - Complete game lifecycle
xxxxxxx Task 1 - Fix duplicate-aware feedback
```

**Important**

This **does not create a third commit**. It simply changes the message of your existing Task 2 commit.

If you have **not pushed yet**, this is completely straightforward. Since your lab requires separate commits for each task, you'll still have exactly **one Task 1 commit + one Task 2 commit**.

next lets do task 3: we need to set difficulty modes by varying the code length, check for symbol range and allowed guesses. We need to make sure that it works for all supported configurations

Worked for 6s

Absolutely. For Task 3, we should first make sure we implement the **difficulty configurations required by your repo**, rather than inventing our own.

I couldn't reliably retrieve the assigned repo's README from the web, so I don't want to guess the exact Easy/Medium/Hard values.

**Please do this first**

Open `README.md` in VS Code and find the **difficulty modes** section. Send me that section (a screenshot is fine).

Once I have the exact specifications, I'll give you **one precise VibeCoding prompt** that will:

- add the required difficulty modes
- vary **code length**
- vary the **allowed symbols/range**
- vary **number of guesses**
- update input validation based on the selected mode
- generate the correct code length/range
- keep the Task 1 duplicate-aware feedback working for every length
- preserve Task 2 history/win/loss behavior
- avoid changing unrelated files

Then we'll test **every supported difficulty**, including boundary/invalid inputs, before making the separate Task 3 commit.

**Don't modify the code yet.** Send me the difficulty requirements from `README.md` first.

### read me says ## Task 3 — Difficulty modes

Add difficulty choices that vary code length, symbol range, and allowed guesses.

The feedback algorithm must work for all supported configurations.

Right — the README specifies **what must vary**, but it does **not specify the actual difficulty values** (for example, Easy = 4 symbols/10 guesses, etc.).

So we should not invent values and pretend they're required.

For Task 3, I recommend we make the difficulty configurations explicit and configurable, while ensuring all three required dimensions vary. A reasonable implementation would be:

Difficulty	Code length	Symbols	Allowed guesses
Easy	3	1–4	12
Medium	4	1–6	10
Hard	5	1–8	8

These are **implementation choices, not README requirements**. If your instructor has given specific values elsewhere, use those instead.

### VibeCoding prompt

Give your coding tool this prompt:

Writing



Implement Task 3 — Difficulty modes in the Mastermind game.

The README requires difficulty choices that vary:

- code length
- symbol range
- allowed guesses

Use these configurations:

- Easy: code length 3, symbols 1–4, 12 allowed guesses
- Medium: code length 4, symbols 1–6, 10 allowed guesses
- Hard: code length 5, symbols 1–8, 8 allowed guesses

### Requirements:

1. At the start of the game, let the player choose Easy, Medium, or Hard.
2. Generate the secret code using the selected configuration.
3. Validate guesses according to the selected code length and symbol range.
4. The selected allowed-guess limit must be enforced.
5. Invalid guesses must not consume a turn or be added to history.
6. Preserve the existing readable guess history from Task 2.
7. Preserve clear win handling and loss handling.
8. Preserve quit behavior.
9. The feedback algorithm from Task 1 must work correctly for every supported code length and symbol range, including duplicate symbols.
10. Do not hard-code feedback for four-digit codes; it must work generically for any configured code length.
11. Do not introduce persistence or external dependencies.
12. Keep the existing `feedback(code, guess)` function interface unchanged.
13. Modify only the files necessary for Task 3. Do not break the functionality completed in Tasks 1 and 2.

After implementing, make sure the game displays the selected difficulty and the corresponding number of turns/symbol range so the player knows the active configuration.

### One important thing

Because Task 3 changes the code length, **don't assume** `exact == 4` means a win anymore.

The win condition needs to become conceptually:

</> Python

```
exact == len(self.code)
```

That is critical for Easy (3 digits) and Hard (5 digits).

Also, the input validation must stop assuming:

</> Python

```
len(raw) != 4
```

It needs to validate against the selected configuration's code length.

After your VibeCoding tool makes the changes, **don't commit yet**.

We'll test:

1. Easy → length 3, symbols 1–4, 12 guesses.
2. Medium → length 4, symbols 1–6, 10 guesses.
3. Hard → length 5, symbols 1–8, 8 guesses.
4. Invalid symbol for each mode.
5. Wrong-length guess for each mode.
6. Duplicate-symbol feedback in at least two different lengths.
7. Win in each mode.
8. Loss at the configured guess limit.
9. Invalid guesses don't consume turns.
10. Quit still works.

That will give us confidence that **Task 1 + Task 2 still work across all Task 3 configurations** before we create the Task 3 commit.

okay lets go ahead

Great. Let's implement **Task 3** now.

Use this as your VibeCoding prompt:

Writing



Implement Task 3 — Difficulty modes in the Mastermind game.

The README requires difficulty choices that vary:

- code length
- symbol range
- allowed guesses

Use these configurations:

- Easy: code length 3, symbols 1–4, 12 allowed guesses
- Medium: code length 4, symbols 1–6, 10 allowed guesses
- Hard: code length 5, symbols 1–8, 8 allowed guesses

Requirements:

1. At the start of the game, let the player choose Easy, Medium, or Hard.
2. Generate the secret code using the selected configuration.
3. Validate guesses according to the selected code length and symbol range.

4. Enforce the selected guess limit.
5. Invalid guesses must not consume a turn or be added to history.
6. Preserve the readable guess history from Task 2.
7. Preserve clear win, loss, and quit handling.
8. The existing duplicate-aware feedback algorithm from Task 1 must work for every supported code length and symbol range.
9. The win condition must be based on the selected code length, not hard-coded to 4.
10. Input validation must use the selected code length and symbol range, not hard-coded values.
11. Keep the existing `feedback(code, guess)` function interface unchanged.
12. Do not introduce persistence or external dependencies.
13. Do not modify `logic.py` unless absolutely necessary; the existing generic feedback implementation should already support different code lengths.
14. Keep Tasks 1 and 2 functionality intact.
15. Display the selected difficulty and its configuration clearly to the player.

Modify only the files necessary for Task 3.

**One correction to keep in mind:** the README tells us that the three properties must vary, but it does **not** prescribe Easy/Medium/Hard numerical values. The `3/4/5`, `1-4/1-6/1-8`, and `12/10/8` values above are our implementation choices, not requirements stated by the README.



Lab\_4\_VibeCoding\_Student\_handout

After your tool applies the changes, **don't commit yet**. Send me the resulting `game.py` (or the tool's change summary), and we'll test all three configurations systematically.

oh no this is not what i meant.. i have given you the exact prompt required for task 3 above because you are my vibing tool

You're right. I misunderstood the workflow. **I am your VibeCoding tool here** — you give me the task requirement, and I should reason through the existing code and give you the actual code changes, not hand you a prompt to use elsewhere.

For Task 3, we'll work directly from your exact requirement:

**Add difficulty choices that vary code length, symbol range, and allowed guesses. The feedback algorithm must work for all supported configurations.**

And we should **not invent difficulty values** that aren't specified by the README.

**First, let's inspect the current implementation**

Before changing anything, I need the current `game.py` after your Task 2 changes. Please paste the current contents of:

</> Plain text

game.py

I already know the current `logic.py` feedback implementation from Task 1, so **don't change `logic.py` yet.**

Once you paste `game.py`, I'll:

1. Identify exactly what needs to change.
2. Give you the Task 3 implementation directly.
3. Keep Tasks 1 and 2 intact.
4. Make the difficulty configuration generic so code length, symbol range, and guess limit aren't hard-coded into the game logic.
5. Then we'll test every supported configuration before committing Task 3.

```
import random
from logic import feedback

class Mastermind:
 def __init__(self):
 self.code = [str(random.randint(1, 6)) for _ in range(4)]
 self.history = []
 self.turns = 10
 self.game_over = False

 def run(self):
 print("Mastermind — enter four digits from 1 to 6.")

 while self.turns > 0 and not self.game_over:
 raw = input(f"{self.turns} turns left > ").strip()

 if raw.lower() == "q":
 self.game_over = True
 print("Game quit.")
 return

 if len(raw) != 4 or any(ch not in "123456" for ch in raw):
 print("Enter exactly four digits from 1 to 6.")
 continue
```

```

guess = list(raw)
exact, partial = feedback(self.code, guess)

self.history.append((raw, exact, partial))
self.turns -= 1

print("Exact:", exact, " Partial:", partial)

print("Guess history:")
for previous_guess, previous_exact, previous_partial in
self.history:
 print(
 f"{previous_guess} -> "
 f"Exact: {previous_exact}, Partial: {previous_partial}"
)

if exact == 4:
 print("Cracked the code!")
 self.game_over = True
 return

if not self.game_over:
 print("Out of turns!")
 print("The code was", "".join(self.code))
 self.game_over = True

```

Yes — now I have the exact current `game.py`, so we can work directly on it.

There is one important thing from the README: it says **difficulty must vary code length, symbol range, and allowed guesses**, but it does **not** prescribe the actual Easy/Medium/Hard numbers.

 Lab\_4\_VibeCoding\_Student\_handout

So rather than silently inventing requirements, I'll make the difficulty settings explicit in one configuration dictionary. For this implementation, we'll use:

- **Easy:** 3 digits, symbols `1-4`, 12 guesses
- **Medium:** 4 digits, symbols `1-6`, 10 guesses
- **Hard:** 5 digits, symbols `1-8`, 8 guesses

These are implementation choices, not claims about the README.

## Replace `game.py` with this

### </> Python

```
import random
from logic import feedback

class Mastermind:
 DIFFICULTIES = {
 "easy": {
 "length": 3,
 "symbols": "1234",
 "turns": 12,
 },
 "medium": {
 "length": 4,
 "symbols": "123456",
 "turns": 10,
 },
 "hard": {
 "length": 5,
 "symbols": "12345678",
 "turns": 8,
```

## Why these changes matter

Your old code had four values hard-coded:

### </> Plain text

```
4 ← code length
1-6 ← symbol range
10 ← turns
exact 4 ← win condition
```

The new version gets all four from the selected difficulty.

Most importantly, the win condition is now:

### </> Python

```
if exact == self.length:
```

So Task 1's `feedback()` doesn't need to know anything about difficulty. It can continue handling 3-, 4-, or 5-character codes generically.

## Now test Task 3

Don't commit yet.

Run:

```
</> powershell
```

```
python main.py
```

Test **Easy** first.

Verify that it says:

```
</> Plain text
```

```
Difficulty: Easy
```

```
Enter 3 digits from 1 to 4.
```

```
You have 12 turns.
```

Then deliberately try:

```
</> Plain text
```

```
1234
```

It should be rejected because it's **4 characters**.

Try:

```
</> Plain text
```

```
555
```

It should be rejected because **5** isn't allowed in Easy.

Then enter a valid 3-digit guess such as:

```
</> Plain text
```

```
123
```

and verify that the turn decreases by exactly one.

After that, test **Medium** and **Hard** the same way.

**Don't commit yet.** Tell me what happens with the Easy test first, and we'll work through all three configurations systematically.

i tested for all easy, medium and hard... when i exceed the limit for easy ie if i exceed 3 it asks me to do it again for 3 valid input..similarly works for medium and hard based on their lengths specified

Perfect.  The **code-length validation works correctly across all three difficulty modes.**

So far for Task 3 we have verified:

-  Easy → accepts exactly 3 characters; rejects longer/shorter guesses.
-  Medium → accepts exactly 4 characters; rejects longer/shorter guesses.
-  Hard → accepts exactly 5 characters; rejects longer/shorter guesses.
-  Invalid input is retried without consuming a turn.

We should do the remaining Task 3 checks before committing.

## Test 1 — Symbol range

For each mode, try a guess containing a symbol outside that mode's range:

**Easy:** try 555

Expected: rejected because Easy allows 1-4 .

**Medium:** try 7777

Expected: rejected because Medium allows 1-6 .

**Hard:** try 99999

Expected: rejected because Hard allows 1-8 .

Make sure the turn count stays unchanged.

## Test 2 — Feedback still works with different lengths

This is particularly important because Task 1 changed the feedback algorithm.

For each difficulty, make a valid guess containing duplicates and check that the Exact/Partial counts don't double-count a code position.

For example, if the generated code happens to contain duplicates, use a guess that also contains duplicates.

## Test 3 — Guess limits

You need to verify the actual limits:

Difficulty	Limit
Easy	12
Medium	10
Hard	8

Use incorrect valid guesses until the final turn.

Expected behavior:

</> Plain text

Out of turns!  
The code was XXXXX

and the game must end.

## Test 4 — Win condition

This catches a very important Task 3 bug.

- Easy must win when **3/3** positions are exact.
- Medium must win when **4/4** are exact.
- Hard must win when **5/5** are exact.

The code should **not** be checking `exact == 4` anymore.

## Test 5 — History

Make at least two valid guesses in each mode and confirm the history contains the guesses with their correct Exact/Partial results.

Once those pass, **Task 3 will be ready for its own commit**. Don't change anything else yet.

yes i checked all the above cases, each win having 3/3 positions exact for easy, 4/4 for medium and 5/5 for hard.. with 12,10 and 8 guesses for easy, medium and hard respectively

Excellent.  **Task 3 is fully verified.**

You've now confirmed all the important requirements:

- ☒ **Easy:** 3-character code, symbols 1-4 , 12 guesses
- ☒ **Medium:** 4-character code, symbols 1-6 , 10 guesses
- ☒ **Hard:** 5-character code, symbols 1-8 , 8 guesses
- ☒ Length validation works for all modes.
- ☒ Symbol-range validation works.
- ☒ Invalid guesses don't consume turns.
- ☒ Guess limits are enforced.
- ☒ Win condition adapts correctly: 3/3, 4/4, 5/5.
- ☒ Task 1's feedback works with the different code lengths.
- ☒ Task 2's game lifecycle/history remains intact.

## Now commit Task 3

First check:

```
</> powershell
```

```
git status
```

Then review exactly what changed:

```
</> powershell
```

```
git diff
```

If the changes are only your Task 3 implementation, run:

```
</> powershell
```

```
git add game.py
```

Then commit:

```
</> powershell
```

```
git commit -m "Task 3 - Add difficulty modes"
```

Finally:

```
</> powershell
```

```
git status
```

and:

```
</> powershell
```

```
git log --oneline -3
```

You should now have **three separate commits**, one for each task:

```
</> Plain text
```

```
Task 3 - Add difficulty modes
Task 2 - Complete game lifecycle
Fix duplicate-aware feedback
```

Don't push yet. Once you show me the `git log --oneline -3` output, I'll verify the commit structure before we move to the final task.

i want to rename task 1

Yes. Since Task 1 is **not the latest commit** now, we shouldn't use `git commit --amend` because that would rename Task 3.

We need to rename the **older Task 1 commit**.

## 1. First see your commit history

Run:

```
</> powershell
```

```
git log --oneline -3
```

You should have something like:

```
</> Plain text
```

```
abc1234 Task 3 - Add difficulty modes
def5678 Task 2 - Complete game lifecycle
ghi9012 Fix duplicate-aware feedback
```

## 2. Rename Task 1

Because it's the third/latest shown commit, use interactive rebase:

```
</> powershell
```

```
git rebase -i HEAD~3
```

An editor will open with something like:

</> Plain text

```
pick abc1234 Task 3 - Add difficulty modes
pick def5678 Task 2 - Complete game lifecycle
pick ghi9012 Fix duplicate-aware feedback
```

Change **only the Task 1 line** from:

</> Plain text

```
pick ghi9012 Fix duplicate-aware feedback
```

to:

</> Plain text

```
reword ghi9012 Fix duplicate-aware feedback
```

Save and close the editor.

Git will then ask for the new commit message. Enter the name you want, for example:

</> Plain text

```
Task 1 - Fix duplicate-aware feedback
```

Save and close again.

### 3. Verify

</> powershell

```
git log --oneline -3
```

You should now have:

</> Plain text

```
Task 3 - Add difficulty modes
Task 2 - Complete game lifecycle
Task 1 - Fix duplicate-aware feedback
```

If you tell me **the exact name you want for Task 1**, I'll give you the exact sequence with that name.

ill change it later how do i exit?

If you're currently in the Git editor and want to **exit without changing anything**, here's what to do depending on what you see:

### If it's Vim

Press:

```
</> Plain text
```

```
Esc
```

Then type:

```
</> Plain text
```

```
:q!
```

and press **Enter**.

That exits without saving.

### If you already started the rebase

After exiting, Git may return you to PowerShell. If you haven't made any changes to the rebase, you're fine.

If you want to **cancel the entire rebase**, run:

```
</> powershell
```

```
git rebase --abort
```

That will return your commits to exactly how they were before you started.

**For your situation, since you want to rename Task 1 later, I'd recommend cancelling the rebase with:**

```
</> powershell
```

```
git rebase --abort
```

Then we can continue with Task 3 normally.

task 1, 2 and 3 is done

Great. 🎉 **Tasks 1, 2, and 3 are complete and verified.**

Your implementation now covers:

### 1. Task 1 — Duplicate-aware feedback

- Exact matches resolved first.
- Partial matches don't reuse already-matched code positions.

### 2. Task 2 — Complete game cycle

- Guess history maintained.
- Invalid input handled without consuming turns.
- Win/loss/quit handled clearly.
- Guess limit enforced.
- Game stops after it ends.

### 3. Task 3 — Difficulty modes

- Easy: 3 characters, 1-4, 12 guesses.
- Medium: 4 characters, 1-6, 10 guesses.
- Hard: 5 characters, 1-8, 8 guesses.
- Feedback works across all configurations.
- Win conditions adapt to the selected code length.

## Before we move on

I'd recommend **not changing the code anymore right now**. We should preserve this verified state.

The next sensible steps are:

**A.** Verify your Git history has 3 separate commits

**B.** Push the completed code to your personal repo

**C.** Record the **after video**

**D.** Make sure your **Lab-4 folder** contains the required videos, updated code, and exported ChatGPT history. The handout explicitly requires separate commits for each task and these deliverables.



Lab\_4\_VibeCoding\_Student\_handout



Lab\_4\_VibeCoding\_Student\_handout

I'd do **A first**, so we know your Git history is clean before pushing.

we are yet left with task 4 that is history and input robustness..  
present a readable guess history and reject malformed guesses

without consuming turns and feedback must correspond to one accepted guess only.

Yes — Task 4 is the final feature set, and importantly, **your current implementation already appears to satisfy much of it** from the testing we've done. We should not blindly rewrite working code.

The Task 4 requirements are:

1. **Readable guess history**
2. **Reject malformed guesses without consuming a turn**
3. **Feedback corresponds to exactly one accepted guess only**

Your current `game.py` already has:

</> Python

```
self.history.append((raw, exact, partial))
```

and displays:

</> Python

```
print("Guess history:")
for previous_guess, previous_exact, previous_partial in self.history:
 print(
 f"{previous_guess} -> "
 f"Exact: {previous_exact}, Partial: {previous_partial}"
)
```

And validation happens **before**:

</> Python

```
guess = list(raw)
exact, partial = feedback(self.code, guess)
self.history.append(...)
self.turns -= 1
```

So malformed input currently doesn't consume a turn or produce feedback.

**The remaining thing we need to explicitly verify**

**One accepted guess → exactly one feedback result.**

For example:

**</> Plain text**

```
10 turns left > 1234
Exact: 1 Partial: 2
```

That guess should produce **one** feedback result and **one** history entry.

If you enter an invalid guess:

**</> Plain text**

```
9 turns left > 9999
Enter exactly ...
```

there should be:

- no **Exact/Partial** output
- no history entry
- still **9 turns left**

Then, when you enter the next valid guess, it should produce exactly one feedback result.

## Before changing anything

Because Tasks 1–3 are already verified, let's treat Task 4 as a **verification/refinement task first**, not a rewrite.

Run the game and perform this exact sequence in one difficulty:

**</> Plain text**

1. Enter a malformed/invalid guess.
2. Confirm the turn count does not decrease.
3. Confirm no Exact/Partial feedback appears.
4. Enter a valid guess.
5. Confirm exactly one Exact/Partial result appears.
6. Enter another valid guess.
7. Confirm both guesses appear separately in history.

Then repeat the malformed-input check in **Easy, Medium, and Hard**, because Task 4 must work with the variable lengths introduced in Task 3.

If all of that passes, **Task 4 may already be complete in your implementation**, and we should avoid unnecessary code changes. Then we'll make the separate Task 4 commit.

i have to commit task 4

Yes. Since Task 4 is verified, let's make the **separate Task 4 commit**.

First check what Git sees:

```
</> powershell
```

```
git status
```

Then review the changes:

```
</> powershell
```

```
git diff
```

If the changes shown are the Task 4 changes, stage them:

```
</> powershell
```

```
git add .
```

Then commit:

```
</> powershell
```

```
git commit -m "Task 4 - Improve history and input robustness"
```

Finally verify:

```
</> powershell
```

```
git status
```

and:

```
</> powershell
```

```
git log --oneline -4
```

Your history should now look roughly like:

```
</> Plain text
```

```
xxxxxxx Task 4 - Improve history and input robustness
xxxxxxx Task 3 - Add difficulty modes
```

```
xxxxxxx Task 2 - Complete game lifecycle
xxxxxxx Task 1 - ...
```

**Don't push yet.** Once you have the `git log --oneline -4` output, send it to me and I'll check that all four tasks are separated correctly before we do the final push.

but no changes have been made since task 4 also has been completed in 1,23,

Exactly. 🍌 You **should not create a fake Task 4 code change just to have a Task 4 commit.**

Since Task 4's requirements were already satisfied by the work in Tasks 1–3, Git correctly has **no new changes** to commit.

But your lab says “**Have Separate Commits for each task.**” So if your instructor expects a visible **Task 4** commit, we need a legitimate, task-related change.

## Best approach

Make a small improvement that is genuinely part of Task 4: improve the history display so it is clearly readable and explicitly numbered.

For example, change:

</> Python

```
print("Guess history:")
for previous_guess, previous_exact, previous_partial in self.history:
 print(
 f"{previous_guess} -> "
 f"Exact: {previous_exact}, Partial: {previous_partial}"
)
```

to:

</> Python

```
print("Guess history:")
for turn, (previous_guess, previous_exact, previous_partial) in enumerate(
 self.history, start=1
):
 print(
 f"{turn}. {previous_guess} -> "
 f"Exact: {previous_exact}, Partial: {previous_partial}"
)
```

Now the history is more explicitly readable:

**</> Plain text**

Guess history:

1. 1234 -> Exact: 1, Partial: 2
2. 5621 -> Exact: 2, Partial: 1
3. 3456 -> Exact: 0, Partial: 2

This is a **real Task 4 improvement**, not an artificial empty commit.

**Then test it**

Run the game, make 2–3 valid guesses, and confirm the history is numbered correctly.

Then check:

**</> powershell**

```
git status
```

You should see `game.py` modified.

Then:

**</> powershell**

```
git add game.py
git commit -m "Task 4 - Improve guess history readability"
```

Finally:

**</> powershell**

```
git log --oneline -4
```

That gives you a legitimate fourth task commit while preserving everything you've already verified.

all the taks have been completed

Yes — **all four tasks are now completed and tested.** 

Your final implementation covers:

- **Task 1:** Duplicate-aware feedback
- **Task 2:** Complete game lifecycle
- **Task 3:** Easy / Medium / Hard difficulty modes

- **Task 4:** Readable history + robust input handling

At this point, **don't make any more functional changes** unless a final verification reveals something wrong.

The remaining lab work is the submission workflow:

1. Verify Git has separate commits for Tasks 1–4.
2. Push your completed code to your personal repository.
3. Record the 10-second after video.
4. Export/save this ChatGPT conversation as your chat history.
5. Put the required **before video, after video, updated code, and chat history** under your **Lab-4** folder, as specified in the handout.  Lab\_4\_VibeCoding\_Student\_handout

The handout also specifically says to **push to your repo and not raise a PR to the main SETAPESU26 repo**, and to have separate commits for each task.  Lab\_4\_VibeCoding\_Student\_handout

If you want, we can now do the **final Git verification + push step-by-step** without touching the code.

i want you to give me one trial for each task so that i can display all the bugs fixed in my after video

Absolutely. For the **after video**, you want a compact demonstration where each task has one visible proof. Since you only need about 10 seconds, we should make the sequence efficient.

## Task 1 — Duplicate-aware feedback

Use a code/guess pair with duplicates.

For example, if your displayed secret is:

</> Plain text

5662

enter:

</> Plain text

5666

Expected:

</> Plain text

Exact: 3 Partial: 0

This demonstrates that the repeated **6** is not incorrectly counted as an extra partial.

## Task 2 — Game lifecycle

Use the secret code shown by the game and enter it as your guess.

For example:

</> Plain text

Secret: 5662

Your guess: 5662

Expected:

</> Plain text

Exact: 4 Partial: 0

Cracked the code!

Then the game should immediately end and return to the terminal.

This visibly demonstrates **win handling + game termination**.

## Task 3 — Difficulty modes

Choose **Hard**.

Show that the game says something like:

</> Plain text

Difficulty: Hard

Enter 5 digits from 1 to 8.

You have 8 turns.

Then enter a valid 5-digit guess.

This single trial demonstrates that:

- code length changed to 5

- symbol range changed to 1–8
- guess limit changed to 8

You don't need to demonstrate all three modes in the 10-second video unless your instructor specifically requires it.

## Task 4 — History + input robustness

Choose a difficulty and first enter an invalid guess.

For example, in Hard:

</> Plain text

8 turns left > 99999

Expected:

</> Plain text

Enter exactly 5 digits from 1 to 8.

The turn count should remain 8.

Then enter a valid guess:

</> Plain text

12345

You should see:

</> Plain text

Exact: ...

Partial: ...

Guess history:

1. 12345 -> Exact: ..., Partial: ...

This demonstrates **invalid input rejection + no turn consumed + readable history + feedback only for the accepted guess**.

## Best after-video sequence

I'd record it in this order:

1. **Task 3:** Select Hard → show 5 digits / 1-8 / 8 turns .
2. **Task 4:** Enter 99999 → rejected, still 8 turns → enter 12345 → feedback + history.
3. **Task 1:** Use a duplicate test such as 5662 / 5666 → Exact: 3, Partial: 0 .
4. **Task 2:** Enter the actual secret → Cracked the code! → game exits.

That gives you a very compact visual demonstration of **all four completed tasks**.

where should i set my breakpoint?

For your **after-video demonstration**, the best breakpoint is in `game.py` at the point where an accepted guess is actually processed.

Set the breakpoint here:

</> Python

```
guess = list(raw)
exact, partial = feedback(self.code, guess)
```

## Why this is the best breakpoint

When you enter a **valid guess**, execution pauses here before feedback is calculated. In VS Code you can then see:

- `self.code` → the secret code
- `guess` → the player's accepted guess
- `self.turns` → remaining turns
- `self.history` → previous guesses

Then press **F10 (Step Over)** and watch:

</> Python

```
exact, partial = feedback(self.code, guess)
```

This is especially useful for demonstrating **Task 1**, because you can show that the duplicate-aware `feedback()` function is being called with the actual code and guess.

## For Task 4

If you want to demonstrate that invalid input **doesn't consume a turn**, put another breakpoint here:

</> Python

```
if (
 len(raw) != self.length
 or any(ch not in self.symbols for ch in raw)
):
```

Enter an invalid guess. The debugger will stop there, and you can show that:

</> Plain text

```
self.turns
```

hasn't changed.

Then press **F5** to continue. The code executes `continue`, so it goes back to the input prompt without reaching:

</> Python

```
self.turns -= 1
```

## One important breakpoint to avoid

Don't put your main breakpoint on:

</> Python

```
self.turns -= 1
```

because by then the validation and feedback have already happened. The earlier breakpoint at:

</> Python

```
exact, partial = feedback(self.code, guess)
```

is much better for showing what's actually happening.

For a **10-second after video**, I'd use just **one breakpoint** at `exact, partial = feedback(...)` and demonstrate the working game normally. The breakpoint is useful if your instructor expects you to show the debugging/vibe-coding process, but it isn't necessary for the final gameplay proof.

```
import random
from logic import feedback
```

```
class Mastermind:
 DIFFICULTIES = {
 "easy": {
 "length": 3,
 "symbols": "1234",
 "turns": 12,
 },
 "medium": {
 "length": 4,
 "symbols": "123456",
 "turns": 10,
 },
 "hard": {
 "length": 5,
 "symbols": "12345678",
 "turns": 8,
 },
 }

 def __init__(self):
 self.history = []
 self.game_over = False
 self.choose_difficulty()

 def choose_difficulty(self):
 print("Choose difficulty:")
 print("1. Easy")
 print("2. Medium")
 print("3. Hard")

 choices = {
 "1": "easy",
 "2": "medium",
 "3": "hard",
 }

 while True:
 choice = input("Enter choice (1-3): ").strip()
```

```
if choice in choices:
 self.difficulty = choices[choice]
 config = self.DIFFICULTIES[self.difficulty]

 self.length = config["length"]
 self.symbols = config["symbols"]
 self.turns = config["turns"]

 self.code = [
 random.choice(self.symbols)
 for _ in range(self.length)
]

 print(
 f"\nDifficulty: {self.difficulty.capitalize()}"
)
 print(
 f"Enter {self.length} digits from "
 f"{self.symbols[0]} to {self.symbols[-1]}."
)
 print(f"You have {self.turns} turns.\n")
 return

print("Invalid choice. Enter 1, 2, or 3.")

def run(self):
 while self.turns > 0 and not self.game_over:
 raw = input(f"{self.turns} turns left > ").strip()

 if raw.lower() == "q":
 self.game_over = True
 print("Game quit.")
 return

 if (
 len(raw) != self.length
 or any(ch not in self.symbols for ch in raw)
):
 print(
 f"Enter exactly {self.length} digits from "
```

```
 f"{self.symbols[0]} to {self.symbols[-1]}."
)
 continue

 guess = list(raw)
 exact, partial = feedback(self.code, guess)

 self.history.append((raw, exact, partial))
 self.turns -= 1

 print("Exact:", exact, " Partial:", partial)

 print("Guess history:")
 for turn, (
 previous_guess,
 previous_exact,
 previous_partial
) in enumerate(self.history, start=1):
 print(
 f"{turn}. {previous_guess} -> "
 f"Exact: {previous_exact}, Partial: {previous_partial}"
)

 if exact == self.length:
 print("Cracked the code!")
 self.game_over = True
 return

 if not self.game_over:
 print("Out of turns!")
 print("The code was", "".join(self.code))
 self.game_over = True
```

Yes — looking at your **current** `game.py`, the breakpoint I recommended is exactly the right place.

## Best breakpoint for Task 1

Put the breakpoint on:

</> Python

```
exact, partial = feedback(self.code, guess)
```

In VS Code, click the red-dot area to the left of that line.

When execution pauses, the Variables panel should show:

</> Plain text

```
self.code
guess
self.turns
self.history
```

For example, if:

</> Plain text

```
self.code = ['5', '6', '6', '2']
guess = ['5', '6', '6', '6']
```

you can step over the `feedback()` call with **F10** and see:

</> Plain text

```
exact = 3
partial = 0
```

That is a very good demonstration of the duplicate-aware fix.

## For Task 4

Your current validation is here:

</> Python

```
if (
 len(row) != self.length
 or any(ch not in self.symbols for ch in row)
):
```

If you want to demonstrate robust input handling, put a breakpoint on that `if`.

Enter an invalid guess, and inspect:

</> Plain text

```
self.turns
```

Then press **F5**. The `continue` sends execution back to the input prompt, and importantly, execution never reaches:

</> Python

```
self.turns -= 1
```

So the turn isn't consumed.

### For Task 3

A useful breakpoint is here:

</> Python

```
self.length = config["length"]
```

You can select Easy/Medium/Hard and show that the configuration changes:

</> Plain text

```
Easy → length=3, symbols=1234, turns=12
Medium → length=4, symbols=123456, turns=10
Hard → length=5, symbols=12345678, turns=8
```

### If you're recording the final demo

I would **not leave the debugger paused throughout the video**. Use the breakpoint only to demonstrate the implementation if your instructor wants debugging evidence. For the actual 10-second after video, run normally and show the working features.

Also, your current `game.py` looks consistent with the Task 3/4 behavior we've been testing. I wouldn't change it further just for the breakpoint.

i want the secret code

Yes — if you want the **secret code for your after-video demo**, you can see it in the debugger.

With your current code, set a breakpoint on:

</> Python

```
self.code = [
 random.choice(self.symbols)
```

```
for _ in range(self.length)
]
```

Then start the game and choose your difficulty. When the debugger pauses, look at:

</> Plain text

```
self.code
```

For example, you might see:

</> Plain text

```
['5', '6', '2', '6']
```

So the secret is:

</> Plain text

```
5626
```

## Even easier

For your demo, put the breakpoint on:

</> Python

```
exact, partial = feedback(self.code, guess)
```

Then **before entering your guess**, the Variables panel will show:

</> Plain text

```
self.code
```

Read the digits from there and enter that exact code.

For example:

</> Plain text

```
self.code = ['5', '6', '2', '6']
```

Enter:

</> Plain text

5626

You'll get:

</> Plain text

Exact: 4 Partial: 0  
Cracked the code!

For **Easy**, look for 3 digits; **Medium**, 4 digits; **Hard**, 5 digits.

couldnt get the code

No problem — let's make this much easier. You **don't need to use a breakpoint at all** to get the secret.

### Easiest method for your video

Temporarily add this line immediately after the code is generated:

</> Python

```
print("DEBUG SECRET:", "".join(self.code))
```

So this section becomes:

</> Python

```
self.code = [
 random.choice(self.symbols)
 for _ in range(self.length)
]

print("DEBUG SECRET:", "".join(self.code))
```

Run:

</> powershell

```
python main.py
```

You'll see something like:

#### </> Plain text

Choose difficulty:

1. Easy
2. Medium
3. Hard

Enter choice (1-3): 2

Difficulty: Medium

Enter 4 digits from 1 to 6.

You have 10 turns.

DEBUG SECRET: 5266

Now you can enter:

#### </> Plain text

5266

and demonstrate:

#### </> Plain text

Exact: 4 Partial: 0

Cracked the code!

## Important

This is **only for your demo/testing**. After you get the secret and finish recording, **remove the debug print** so you don't leave debugging output in your submitted code.

If you want, there's an even better way to make the **after-video demonstrate all 4 tasks in one short run** without adding any debug code.

okay, it is working now.. i have recorded the final video

Perfect! 🎉 Your **after video is done**.

At this point, your coding work is essentially complete. Before submission, let's just make sure the final project state is clean:

1. ☒ Task 1 completed and tested
2. ☒ Task 2 completed and tested

3. ☒ Task 3 completed and tested
4. ☒ Task 4 completed and tested
5. ☒ Before video recorded
6. ☒ After video recorded

The remaining submission steps are:

- **Verify the four Git commits**
- **Remove the temporary `DEBUG SECRET` line** if it is still in `game.py`
- **Run one final `git status`**
- **Push to your personal GitHub repository**
- **Prepare the `Lab-4` folder with:**
  - before video
  - after video
  - updated code
  - exported ChatGPT chat history

The handout specifically requires the videos, updated code, and exported chat history, and says to push the code with separate commits for each task.

 Lab\_4\_VibeCoding\_Student\_handout

 Lab\_4\_VibeCoding\_Student\_handout

**Don't make any more code changes unless we find the debug line.** The safest next step is to check your final `git status` and commit history before pushing.

what do i need to submit now?

According to your **Lab 4 handout**, you need to submit **three things** under your `Lab-4` folder:

 Lab\_4\_VibeCoding\_Student\_handout

### 1. 🎥 Before video

Your ~10-second recording from **before making changes**, showing the original/broken game.

☒ You already recorded this.

### 2. 🎥 After video

Your ~10-second recording **after all changes**, demonstrating the fixed features.

☒ You just recorded this.

### 3. 💻 Updated code

Your completed Mastermind project/code after Tasks 1–4.

This is the code containing your fixes and difficulty modes.

**Also required: Chat history**

The handout specifically says the deliverables include:

**"Chat history exported as a doc/pdf"**



Lab\_4\_VibeCoding\_Student\_handout

So export/save this ChatGPT conversation as a **document or PDF** and put it in the **Lab-4** folder as well.

Your **Lab-4** folder should therefore contain

</> Plain text

Lab-4/

├─ Before Video

├─ After Video

├─ Updated Code

└─ Chat History.pdf

The handout also requires you to **push your code to your personal repo** and have **separate commits for each task**. It explicitly says **not to raise a PR to the main SETAPESU26 repo**.



Lab\_4\_VibeCoding\_Student\_handout

So your final checklist is:

- ☒ Before video
- ☒ After video
- ☒ Tasks 1–4 completed
- ☐ Verify 4 Git commits
- ☐ Remove temporary **DEBUG SECRET** line if still present
- ☐ Push to your personal GitHub repo
- ☐ Export ChatGPT conversation as PDF/doc
- ☐ Put the required files in **Lab-4**

**One thing I would do before submitting:** run `git status` and check whether the temporary debug line is still in `game.py`.

check-> i have before and after video, updated code.. committed all tasks and pushed everything, ill export the chat history

Yes — you've completed the actual lab work.

Your submission checklist is now:

- **Before-change video**
- **After-change video**

-  Updated code
-  All 4 tasks committed separately
-  Everything pushed to your personal GitHub repo
-  Chat history exported as PDF/doc

Once you export the chat history, you should have **all required deliverables**.

One last thing before submitting: if you temporarily added `DEBUG SECRET:` for the video, make sure that line has been removed from your final code. Otherwise, you're essentially done. 🎉